



UNIVERSIDAD
DE MÁLAGA



Escuela de Ingenierías Industriales

Ingeniería de Sistemas y Automática

Trabajo de Fin de Grado

Control de Motores de Robot de Rescate Donatello

Donatello Rescue Robot Motor Control

Grado en Ingeniería Electrónica Industrial

Autor: Mercedes Román Ruiz

Tutor: Antonio José Muñoz Ramírez

Málaga, Mayo de 2024

DECLARACIÓN DE ORIGINALIDAD DEL PROYECTO/TRABAJO FIN DE GRADO

D.Dña. Mercedes Román Ruiz.

DNI/Pasaporte: 77228475H. Correo electrónico: mercedesromanruiz@gmail.com.

Titulación: Grado en Ingeniería Electrónica Industrial.

Título del Proyecto/Trabajo: Control de Motores de Robot de Rescate Donatello.

DECLARA BAJO SU RESPONSABILIDAD

Ser autor/a del texto entregado y que no ha sido presentado con anterioridad, ni total ni parcialmente, para superar materias previamente cursadas en esta u otras titulaciones de la Universidad de Málaga o cualquier otra institución de educación superior u otro tipo de fin.

Así mismo, declara no haber trasgredido ninguna norma universitaria con respecto al plagio ni a las leyes establecidas que protegen la propiedad intelectual, así como que las fuentes utilizadas han sido citadas adecuadamente.

En Málaga, a 27 de Mayo de 2024.

Fdo.: Mercedes Román Ruiz

Control de Motores de Robot de Rescate Donatello

- **Autor:** Mercedes Román Ruiz.
- **Tutor:** Antonio José Muñoz Ramírez.
- **Departamento:** Ingeniería de Sistemas y Automática.
- **Titulación:** Grado en Ingeniería Electrónica Industrial.
- **Palabras clave** Vehículo Robótico, Motor BLDC, Arduino Mega 2560, Arduino MKR, CAN Bus, Serial, MATLAB, Simulink, ROS, Control.

Resumen

Este Trabajo de Fin de Grado consiste en el desarrollo e implementación de un controlador para los motores del vehículo robótico de rescate Donatello, perteneciente al equipo de robótica RoboRescueUMA de la Universidad de Málaga. La implementación se realiza en una placa Arduino Mega 2560 con conexión y supervisión a través de un ordenador de abordo compacto Nuc Intel i7 con sistema operativo Windows.

Los servomotores utilizados, GIM8108v3, del fabricante SteadyWin se caracterizan por ser motores de corriente continua sin escobillas controlados a través de órdenes recibidas a través de un bus CAN. Esta comunicación se ha realizado haciendo uso de una *shield* basada en el microchip MCP2515 conectada a la placa Arduino.

Entre otros, se han desarrollado dos modelos de Simulink para el control del vehículo. El primero se ejecutará en tiempo real en la placa Arduino Mega 2560, calculará los parámetros de control de cada rueda y los transmitirá a los motores. El segundo modelo, que se ejecuta en el ordenador de abordo, servirá como interfaz para el usuario, permitiendo el control de la velocidad lineal y angular del vehículo, por ejemplo a través de un mando de Xbox conectado por Bluetooth al ordenador. Ambos modelos intercambiarán información a través de una comunicación serial establecida entre el ordenador y la placa Arduino.

Donatello Rescue Robot Motor Control

- **Author:** Mercedes Román Ruiz.
- **Tutor:** Antonio José Muñoz Ramírez.
- **Department:** System and Automatic Engineering.
- **Degree:** Industrial Electronic Engineering.
- **Keywords** Robotic Vehicle, BLDC Motor, Arduino Mega 2560, Arduino MKR, CAN Bus, Serial, MATLAB, Simulink, ROS, Control.

Abstract

This Bachelor's Thesis consists of the development and implementation of a controller for the motors of the Donatello robotic rescue vehicle, belonging to the RoboRescueUMA robotics team at the University of Malaga. The implementation is carried out on an Arduino Mega 2560 board with connection and monitoring through a compact onboard computer, an Intel Nuc i7 with Windows operating system.

The servomotors used, GIM8108v3 from the manufacturer SteadyWin, are characterized by being brushless direct current motors controlled through commands received via a CAN bus. This communication has been carried out using a shield based on the MCP2515 microchip connected to the Arduino board.

Among other things, two Simulink models have been developed for vehicle control. The first will run in real-time on the Arduino Mega 2560 board, calculating the control parameters for each wheel and transmitting them to the motors. The second model, running on the onboard computer, will serve as a user interface, allowing control of the vehicle's linear and angular speed, for example, through an Xbox controller connected via Bluetooth to the computer. Both models will exchange information through a serial communication established between the computer and the Arduino board.

*A mi madre, mi guía en lo personal,
y a mi padre, mi referente en lo profesional.*

Mercedes Román Ruiz



Agradecimientos

Agradecer a mi tutor, Antonio José Muñoz Ramírez, su tiempo y ayuda durante la realización de este proyecto.

Gracias a Pablo Arroyo Suarez, por todos los días de trabajo compartidos en el taller y la amistad que ha surgido.

A mi familia, por su apoyo en estos años de carrera.

A mi pareja, por creer siempre en mí.

Glosario de Términos

Bloque Un bloque en Simulink es una entidad gráfica que representa una operación, función o elemento específico dentro de un modelo, utilizado para construir y simular sistemas dinámicos.

Controlador Programa que permite a una computadora u otro dispositivo electrónico manejar los componentes que tiene instalados.

Controlador de Potencia Dispositivo que regula y gestiona el flujo de energía eléctrica a un sistema o componente, asegurando un funcionamiento eficiente y seguro.

Datos Estándar Los Datos Estándar son aquellos que ocupan 11 bits en el protocolo CAN y se utilizan principalmente para la comunicación entre dispositivos en sistemas automotrices.

Datos Extendidos Los Datos Extendidos ocupan 29 bits y se emplean para transmitir datos en redes más complejas y grandes, como en aplicaciones industriales y de automatización.

Firmware Software integrado en dispositivos electrónicos que controla sus funciones básicas y proporciona una interfaz entre el hardware y el software.

Hardware Conjunto de aparatos de una computadora.

Microcontrolador Pequeño ordenador integrado en un solo chip que contiene un procesador, memoria y periféricos, utilizado para controlar dispositivos electrónicos.

Middleware Software que actúa como intermediario facilitando la comunicación y gestión de datos entre diferentes aplicaciones o sistemas.

Modelo Un modelo en Simulink es una representación visual de un sistema dinámico o proceso, creado mediante bloques funcionales interconectados que describen su funcionamiento y relaciones.

Motor BLDC Motor eléctrico de corriente continua sin escobillas.

Paquetes Informáticos Conjunto de programas diseñados para realizar tareas específicas en un ordenador.

Periférico Aparato auxiliar e independiente conectado a la unidad central de una computadora u otro dispositivo electrónico.

Servomotores Sistema electromecánico que amplifica la potencia reguladora.

Shield Módulo de expansión que se conecta a una placa para añadirle funcionalidades adicionales, como conectividad, sensores o controladores.

Software Conjunto de programas, instrucciones y reglas informáticas para ejecutar ciertas tareas en una computadora.



UNIVERSIDAD
DE MÁLAGA



Acrónimos

BLDC	BrushLess Direct Current
CAN	Controller Area Network
CI	Circuito Integrado
ID	Identificador
IDE	Integrated Ddevelopment Environment
MCD	Modelo Cinemático Directo
MCI	Modelo Cinemático Inverso
MCU	Microcontroller Unit
PC	Personal Computer
PWM	Pulse Width Modulation
ROS	Robot Operating System
RX	Recibir
SPI	Serial Peripheral Interface
TTL	Transistor-Transistor Logic
TX	Transmitir
UART	Universal Asynchronous Receiver-Transmitter
USB	Universal Serial Bus



UNIVERSIDAD
DE MÁLAGA



Índice general

1. Introducción	11
1.1. Objeto	11
1.2. Antecedentes	11
1.2.1. Robótica de Rescate en la UMA	11
1.2.2. Motores GIM8108v3 y el MIT Mini Cheetah	13
1.3. Objetivo	14
1.4. Fases del trabajo	15
1.5. Contenido de la memoria	15
2. Componentes del Hardware	17
2.1. Introducción	17
2.2. Motores sin escobillas	17
2.3. Sistema Electrónico con Arduino Mega	19
2.3.1. Arduino Mega 2560	20
2.3.2. CAN-BUS Shield v1.2	21
2.4. Sistema Electrónico con Arduino MKR	23
2.4.1. Arduino MKR WiFi 1010	23
2.4.2. Arduino MKR CAN Shield	25
2.5. Chip MCP2515	26
2.5.1. <i>SPI - Serial Peripheral Interface</i>	26
2.5.2. <i>CAN - Controller Area Network</i>	27

3. Software utilizado	29
3.1. Introducción	29
3.2. Arduino IDE	29
3.3. TeraTerm	30
3.4. MATLAB	32
3.5. Simulink	33
3.6. GitHub	34
3.7. ROS	35
4. Control de motores GIM8108v3	37
4.1. Introducción	37
4.2. Firmware 1.9	37
4.3. Modelo Simulink	39
4.3.1. Bloques CAN	39
4.3.2. Subsistema comunicación motores	45
4.4. Arduino MKR	49
5. Introducción a ROS	51
5.1. Introducción	51
5.2. Software Adicional	51
5.2.1. VMware Workstation 17 Player	51
5.3. Configuración	53
5.4. Pruebas	55
5.4.1. Velocidad	56
5.4.2. Posición	56
5.4.3. Lidar	56
6. Control PC	59
6.1. Introducción	59
6.2. Pruebas iniciales comunicación serie.	59
6.3. Subsistemas Simulink	60
6.3.1. <i>Send to Arduino</i>	60
6.3.2. <i>Receive from Arduino</i>	62

6.3.3. <i>XBox Controller</i>	63
6.3.4. <i>ROS Publisher</i>	65
6.4. Modelo control PC	65
7. Control Vehículo	67
7.1. Introducción	67
7.2. Subsistemas Simulink	67
7.2.1. <i>Receive from PC</i>	67
7.2.2. Modelo cinemático inverso	68
7.2.3. Control Ruedas	69
7.2.4. Modelo cinemático directo	71
7.2.5. <i>Send to PC</i>	72
7.2.6. Odometría	73
7.3. Modelo control vehículo	74
8. Conclusiones y mejoras futuras	77
8.1. Introducción	77
8.2. Conclusiones	77
8.3. Posibles mejoras	78
Bibliografía	79
A. ComunicacionMotores.ino	79
B. CAN Master Test- Ben Katz	81
C. MATLAB Functions	87



UNIVERSIDAD
DE MÁLAGA



Índice de figuras

1.1. Robots del Laboratorio de Robótica y Mecatrónica RoboticalSA	12
1.2. Esquema conexión Arduino Mega 2560 y módulo MCP2515. TFG-EZ	13
1.3. Motores de MIT y SteadyWin	13
1.4. MIT Mini-Cheetah	14
1.5. Chasis antiguo y actual del vehículo robótico.	15
2.1. Motor GIM8108 v3	18
2.2. Esquema de conexiones del vehículo.	20
2.3. Arduino Mega 2560 ArduinoMega2560rev3	21
2.4. CAN-BUS Shield v1.2 ArduinoMegaCANShield	21
2.5. CAN-BUS Shield v1.2 ArduinoMegaCANShield - Descripción General . . .	22
2.6. Arduino MKR WiFi 1010 ArduinoMKRWiFi1010	23
2.7. Arduino MKR CAN Shield ArduinoMKRCANShield	25
2.8. MCP2515 - Microchip Technology MCP2515	26
2.9. Conexión bus SPI Wiki-SPI	27
2.10. Conexión bus CAN Wiki-CAN	28
2.11. Niveles de tensión del CAN Bus Wiki-CAN	28
3.1. Logotipo Arduino. Arduino	29
3.2. Logotipo TeraTerm. TeraTerm	30
3.3. Convertidor de USB a TTL UART. TTL	30
3.4. Menu (TeraTerm).	31

3.5. Setup (TeraTerm).	32
3.6. Logotipo MATLAB MATLAB	32
3.7. Logotipo Simulink Simulink	33
3.8. Contenido proyecto en Simulink	34
3.9. Logotipo GitHub Wiki-GitHub	34
3.10. Reposito del proyecto en GitHub	35
3.11. Logotipo ROS ROS	35
3.12. Ejemplo red ROS MathWorksROS	35
4.1. Bloques CAN	40
4.2. CAN Pack - Block Parameters	40
4.3. Estructura del mensaje CAN enviado al motor.	42
4.4. CAN Unpack - Block Parameters	43
4.5. Estructura del mensaje CAN recibido desde el motor.	44
4.6. Contenido del subsistema Motores.	45
4.7. Transmisión de datos a los motores.	46
4.8. Bloques utilizados.	47
4.9. Recepción de datos de los motores.	48
5.1. Logotipo VMware VMware	51
5.2. Logotipo Ubuntu Ubuntu	52
5.3. Escritorio de la máquina virtual ros_noetic_foxy_gazebov11.vmx	52
5.4. Logotipo Gazebo Gazebo	53
5.5. Configuración red.	54
5.6. Diagrama de variables de entorno GazeboMatlab	55
5.7. Gazebo House	55
5.8. Comparación datos lidar y simulador.	57
6.1. Modelo prueba serie Arduino.	59
6.2. Modelo prueba serie PC.	60
6.3. Subsistema Send to Arduino	61
6.4. Parámetros de los bloques	61
6.5. Subsistema Receive from Arduino	62
6.6. Parámetros del bloque Serial Receive	63



6.7. Mando inalámbrico XBox	63
6.8. Subsistema XBox Controller	64
6.9. Contenido bloque ‘ <i>chart</i> ’	64
6.10. Subsistema ROS Publisher	65
6.11. Modelo Control PC	66
7.1. Subsistema <i>Receive from PC</i>	67
7.2. Parámetros de los bloques.	68
7.3. Subsistema Modelo Cinemático Inverso	69
7.4. Subsistema Control Ruedas	69
7.5. Subsistema Parámetros Modo	70
7.6. Subsistema Control Kp	70
7.7. Subsistema Ruedas	71
7.8. Subsistema Modelo Cinemático Directo	72
7.9. Subsistema Send to PC	72
7.10. Parámetros de los bloques.	73
7.11. Subsistema Odometría	74
7.12. Modelo Control Vehículo	75



UNIVERSIDAD
DE MÁLAGA



Índice de tablas

2.1. Características del motor GIM8108 v3	19
2.2. Especificaciones técnicas del Arduino MKR	24
2.3. Especificaciones técnicas del Arduino MKR Shield	25
3.1. Tabla Comparativa ROS y ROS 2.	36



Capítulo 1

Introducción

1.1 Objeto

Este Trabajo de Fin de Grado pretende poner de manifiesto los conocimientos adquiridos en el Grado de Ingeniería Electrónica Industrial. Dicho trabajo permite profundizar en alguna/s de las materias desarrolladas en el Grado.

1.2 Antecedentes

La Robótica de Rescate es un campo emocionante y vital que fusiona la tecnología robótica con el propósito humano de salvar vidas en situaciones de emergencia. Con un enfoque en el diseño, desarrollo y despliegue de robots inteligentes y ágiles, esta disciplina busca proporcionar soluciones innovadoras para enfrentar desafíos en entornos peligrosos o inaccesibles para los equipos de rescate convencionales. Desde terremotos hasta incendios forestales, la Robótica de Rescate ofrece herramientas y sistemas avanzados para mejorar la eficiencia y seguridad en operaciones de salvamento, protegiendo tanto a los rescatistas como a las víctimas.

1.2.1 Robótica de Rescate en la UMA

Entrando en proyectos llevados a cabo en la Universidad de Málaga, encontramos el Laboratorio de Robótica y Mecatrónica, integrado en el Departamento de Ingeniería de Sistemas y Automática (ISA). Entre sus actividades de investigación encontramos la Robótica para catástrofes, búsqueda y rescate y la Robótica de campo.



(a) Cuádriga



(b) Rambler

Figura 1.1: Robots del Laboratorio de Robótica y Mecatrónica **RoboticaISA**.

Cuádriga es un robot que cuenta con dirección deslizante, un sistema de control avanzado que se utiliza para mejorar la maniobrabilidad y la estabilidad del vehículo en diversos terrenos. Utiliza motores de corriente continua con escobillas con engranajes de reducción lo cual le permite ejercer un gran par a baja velocidad, con la contraprestación de no poder alcanzar velocidades elevadas.

Rambler(DPI2011-22443) (Figura 1.1b), es un robot explorador del tamaño y la potencia de un vehículo eléctrico, que alcanza una velocidad de 80 kilómetros por hora. Con un peso aproximado de 200 kg, consta de un sistema de amortiguación activa y un brazo manipulador que permite realizar el triaje a las víctimas. Este, se presentó de manera oficial en las VIII Jornadas de Seguridad, Emergencias y Catástrofes acogida en la Escuela de Industriales en el año 2014.

El sistema de tracción está compuesto por ruedas dotadas de motores de accionamiento directo de corriente continua sin escobillas, más eficientes que los motores del robot. Estos motores permiten una gran velocidad pero presentan el inconveniente de carecer de par necesario para el movimiento a velocidades muy bajas.

Tanto el robot Rambler como el robot Cuádriga, carecen de electrónica de potencia y electrónica de control, que en ambos casos presentan la necesidad de un volumen de espacio considerable dentro del vehículo, a parte del coste adicional que conlleva.

Donatello v2021

El robot utilizado en este proyecto, Donatello, es un robot de rescate con el que se participa en la competición RoboCupRescue Robot League. Esta es una liga internacional de equipos con un objetivo: desarrollar y demostrar capacidades robóticas avanzadas para equipos de rescate utilizando competiciones anuales para evaluar y campamentos de enseñanza para difundir las mejores soluciones robóticas.

En el año 2021 se realizó el primer diseño del robot de rescate Donatello. Este fue realizado por Eva María Góngora Rodríguez **TFG-EV** y Ezequiel Farina **TFG-EZ**. Este diseño presentaba algunas deficiencias, ya que el excesivo cableado y las vibraciones del vehículo provocan

la desconexión de cables y por tanto fallos.

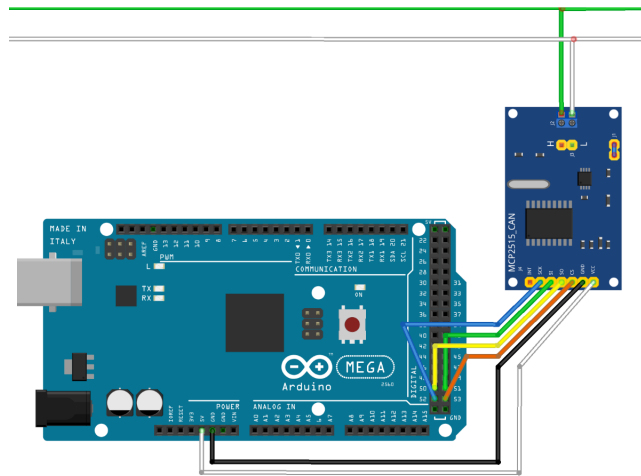


Figura 1.2: Esquema conexión Arduino Mega 2560 y módulo MCP2515. **TFG-EZ**

La figura 1.2 muestra el esquema existente anterior a este proyecto. Se plantea por tanto la sustitución del módulo CAN por una *shield* para el Arduino Mega 2560. Esto hará que el sistema electrónico del vehículo robótico sea más robusto. Reduciendo el número de cables y por tanto el riesgo de una desconexión de estos. También, se estudiará la sustitución de la placa Arduino Mega 2560 por una Arduino MKR con CAN Shield.

1.2.2 Motores GIM8108v3 y el MIT Mini Cheetah

Los motores utilizados en este proyecto (Figura 1.3a), cuyas características se desarrollan en más profundidad en el capítulo sobre el hardware utilizado, se basan en los del MIT Mini Cheetah (Figura 1.3b) cuyo proyecto fue liderado por Benjamin Katz **Katz2016**.



(a) Motor GIM8108v3 **GIM8108v3**



(b) Motor MIT **Katz2016**

Figura 1.3: Motores de MIT y SteadyWin

1.3. OBJETIVO

Este robot cuadrúpedo de algo más de 9 kg, destaca por ser robusto, ligero y ágil. Llama la atención su capacidad de levantarse en caso de caer al suelo como si nada hubiera pasado, aparte de ser el primer ‘robot acróbata’ capaz de hacer volteretas.

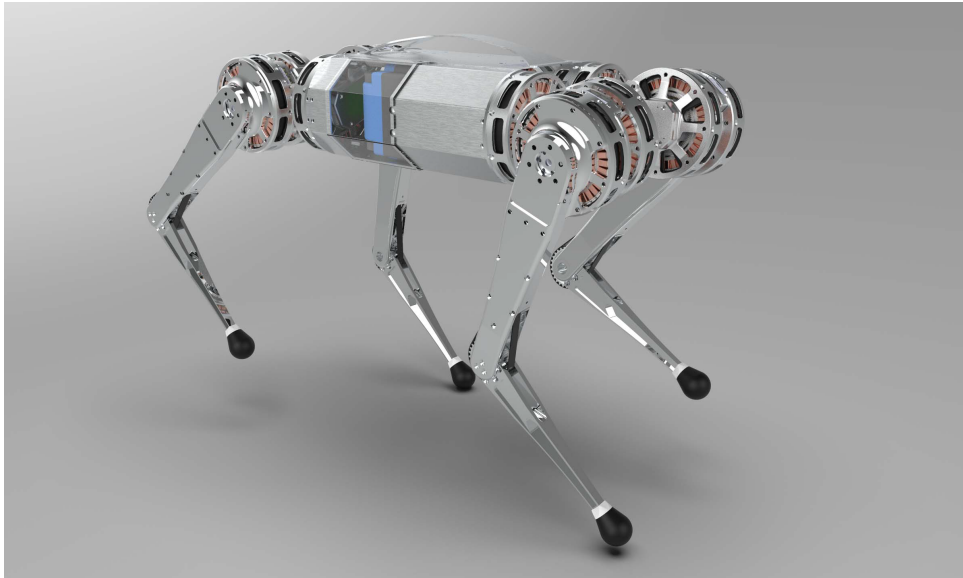


Figura 1.4: MIT Mini-Cheetah

Su actuador utiliza el rotor y el estator de un motor sin escobillas comercial diseñado para grandes drones RC. El modelo en particular usado es casi idéntico en forma y rendimiento al T-Motor U8, que se utilizó en el robot Minitaur. Estos motores están integrados en un actuador que también incluye una reductora planetaria de una sola etapa de 6:1 y la electrónica de potencia y de control del motor con sensor de posición incorporado.

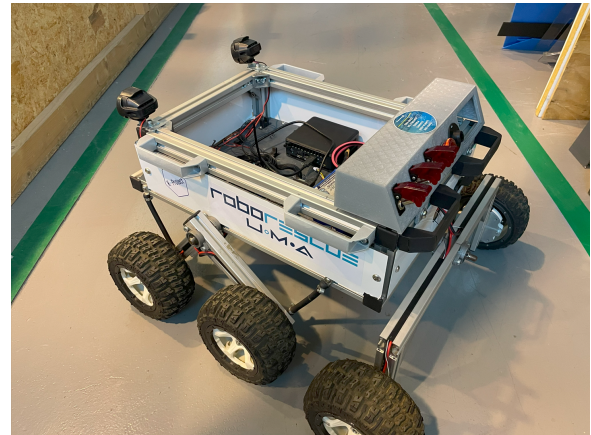
Steadywin ofrece una versión de estos motores, pudiendo elegir entre el controlador de código abierto del MIT (utilizado en este proyecto) y un nuevo controlador SHS.

1.3 Objetivo

El objetivo de este trabajo es implementar, haciendo uso de SIMULINK y una placa Arduino, un controlador para los servomotores del vehículo robótico Donatello. Este robot forma parte del equipo de RoboRescue de la Universidad de Málaga.



(a) Donatello año 2021 TFG-EV.



(b) Donatello año 2024.

Figura 1.5: Chasis antiguo y actual del vehículo robótico.

1.4 Fases del trabajo

El trabajo tendrá lugar en el taller 0.27.T.II y constará de las siguientes partes:

1. Instalación del software y búsqueda bibliográfica: instalación del software que se va a utilizar, como MATLAB, Arduino IDE y SIMULINK, así como de los paquetes específicos necesarios.
2. Estudio de los motores.
3. Desarrollo del código de control: se modelará utilizando Simulink.
4. Realización de pruebas: se comprobará el correcto funcionamiento del modelo diseñado experimentalmente.
5. Documentación: elaboración de la memoria del trabajo realizado así como de la documentación técnica necesaria.

1.5 Contenido de la memoria

El documento se divide en ocho capítulos, siendo este capítulo de introducción el primero de la memoria. A continuación se describen brevemente los siete capítulos restantes.

- El segundo capítulo se introducirán los componentes del hardware utilizados.
- En el tercer capítulo se describe el software utilizado.
- El cuarto capítulo explica el control del motor de SteadyWin. Entrando en detalles de la comunicación CAN y su firmware.

- El quinto capítulo sirve de introducción a ROS. Comentando en que consiste este middleware y probando su conexión con MATLAB y su correcto funcionamiento simulado en Gazebo.
- En el sexto capítulo se describen los subsistemas que integran el modelo de control del vehículo desde el ordenador. Este capítulo se inicia con una introducción a la comunicación en serie ordenador-placa, para posteriormente adentrarse en la explicación de todos los subsistemas que hacen posible este control.
- A continuación, en el séptimo capítulo describe los subsistemas que integran el modelo del control del vehículo. Desde su modelo cinemático inverso a modelo cinemático directo, pasando por odometría o control de parámetros.
- Por último, en el octavo capítulo, se elaboran las conclusiones obtenidas durante la realización de este trabajo, así como la proposición de mejoras futuras.

Finalmente, se incluye una sección de anexos donde se recogen los distintos códigos usados en este proyecto.

Capítulo 2

Componentes del Hardware

2.1 Introducción

Un vehículo robótico típicamente consta de varias partes esenciales que trabajan en conjunto para su funcionamiento. Se diferencian partes mecánicas y electrónicas. En el sistema mecánico podemos encontrar el chasis, que proporciona la estructura física sobre la cual se montan todos los componentes. Ya en el sistema electrónico encontramos: sensores, que permiten al vehículo percibir su entorno; actuadores, responsables de ejecutar acciones físicas como respuesta a señales recibidas; unidad de control, encargada del procesamiento de datos y toma de decisiones basadas en algoritmos; una fuente de energía, que permita alimentar el vehículo; y por último un sistema de comunicación, que facilite la interacción del vehículo con otros dispositivos.

En este capítulo se describirán los componentes principales del hardware que han sido utilizado en el desarrollo de este proyecto. Comentando sus características y el uso que se hace de los mismos. El enfoque principal estará en los componentes empleados para la comunicación, el control y el actuador.

2.2 Motores sin escobillas

En la actualidad, podemos diferenciar cinco tipos de motores en función del tipo de energía que utilizan, encontrando motores de gasolina, diésel, eléctricos, híbridos y de gas. Siendo la tendencia la utilización de motores eléctricos. Dentro de estos podemos encontrar varios tipos, aunque los más utilizados son los Paso a Paso, *Brushed* y *Brushless*.

El motor eléctrico basa su funcionamiento en la capacidad de convertir energía eléctrica en energía cinética, y para este efecto se sirve de la energía magnética por la que se repelen dos campos. Los motores de corriente continua sin escobillas, *Brushless Direct Current (BLDC)*, no emplean escobillas para el cambio de polaridad en el rotor. La ausencia de escobillas hace

que tenga una vida útil mayor requiriendo bajo mantenimiento. Además, debido a la falta de fricción en el interior del motor hace que tenga una mayor potencia, velocidad y eficiencia, y que por la misma razón produzca un menor ruido.

En este proyecto se utilizarán los servomotores BLDC GIM8108 V3 de SteadyWin (Figura 2.1), caracterizados por ser motores de corriente continua sin escobillas que utilizan el protocolo de comunicación CAN bus.

SteadyWin®
Intelligence Drive the Future

GIM8108
MIT机械狗电机



超小体积，全封闭设计
内置开源驱动，6:1高精度行星减速
广泛应用于机器狗、四足机器人、外骨骼机器人和工业机械臂等

Figura 2.1: Motor GIM8108 v3

La falta de documentación por parte del fabricante dificultó el inicio del proyecto. A partir de uno de estos ficheros **GIM8108v3** se obtiene los parámetros del motor.

Parámetros del motor GIM8108v3		
Alimentación	Voltaje	$24VDC \pm 10 \%$
	Intensidad	7A
Detalles	Torque (par)	$1,27Nm$
	Velocidad de giro	1500rpm
	Velocidad máxima	2000rpm
	Potencia	200W
Señal de realimentación		Encoder absoluto de 15 bits, un giro representa 32768
Relación de transmisión		6 : 1
Torque de parada en funcionamiento		$15Nm$
Modo de refrigeración		Refrigeración natural
Peso		600g
Control de posición	Frecuencia de muestreo	2kHz
Protección		Aviso de bloqueo del rotor
Interfaz de comunicación		Smartcan (CAN, 1MHz)
Entorno de trabajo	Temperatura ambiente	0 – 40°C
	Temperatura máxima permitida	85°C
	Humedad	5 – 95 %

Tabla 2.1: Características del motor GIM8108 v3

2.3 Sistema Electrónico con Arduino Mega

El siguiente esquema (Figura 2.2) se ha realizado con Fritzing. En su realización se ha seguido el código de colores empleado en el vehículo, a excepción de la señal CAN_H, cuyo color gris apenas se diferenciaba del color blanco de la señal CAN_L. Los motores GIM8108 utilizados en este proyecto no se encuentran modelados en Fritzing por lo que se representan mediante un conjunto Motor BLDC - Controlador de Potencia. Por último, las baterías y la *shield* del Arduino, se han reemplazado por unas de iguales características pero de aspecto y marca distinta.

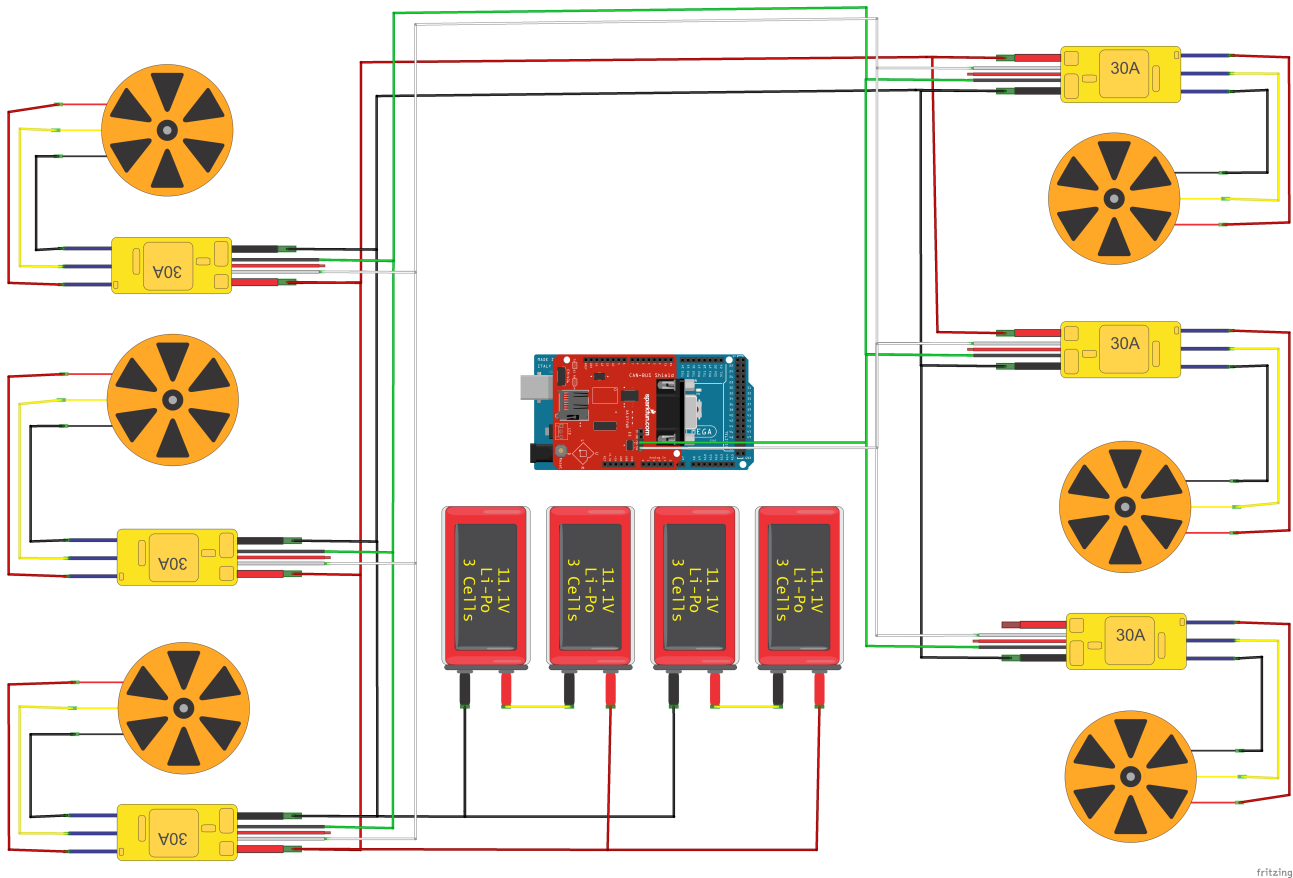


Figura 2.2: Esquema de conexiones del vehículo.

2.3.1 Arduino Mega 2560

Esta placa es parte de la familia de placas Mega, las cuales destacan por su uso en proyectos que demandan mucha potencia informática. Además posee un mayor número de puertos de entrada/salida, interfaces e interrupciones que otras placas de la familia. Esto y su soporte en Simulink, es el motivo por el cual se utiliza el Arduino Mega 2560 para este proyecto.

La placa Arduino Mega 2560 es una actualización que reemplaza a la placa Arduino Mega. Esta nueva placa está basada en el microcontrolador ATmega2560. Posee 54 pines de entrada/salida digitales (15 de ellos pueden ser utilizados para salidas de PWM), 16 entradas analógicas, 4 UARTS (puertos seriales), un cristal oscilador de 16 MHz, conector USB, puerto de alimentación, conector ICSP y botón de reset.



Figura 2.3: Arduino Mega 2560 `ArduinoMega2560rev3`

2.3.2 CAN-BUS Shield v1.2

Esta *shield* funciona perfectamente con las placas Arduino UNO (ATmega328), Arduino Mega (ATmega1280/2560) y Arduino Leonardo (ATmega32U4).



Figura 2.4: CAN-BUS Shield v1.2 `ArduinoMegaCANShield`

Características:

- Implementa CAN V2.0B hasta 1Mb/s
- Interfaz SPI hasta 10MHz
- Marcos de datos y remotos estándar (11 bits) y extendidos (29 bits)
- Dos búferes de recepción con almacenamiento de mensajes priorizados

- Conector sub-D de 9 pines estándar industrial

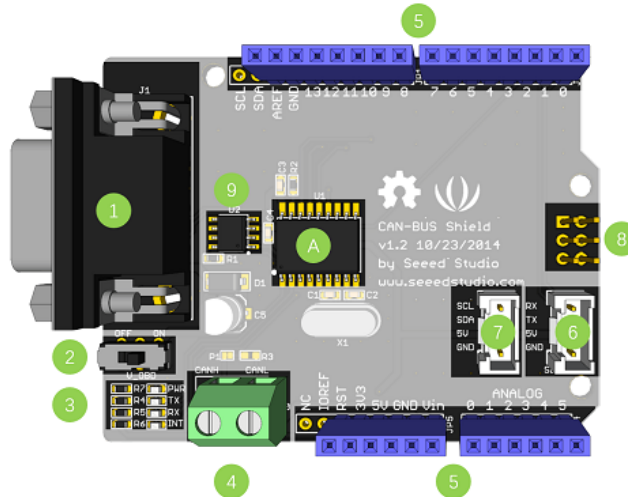


Figura 2.5: CAN-BUS Shield v1.2 **ArduinoMegaCANShield** - Descripción General

Descripción General:

1. Interfaz DB9: Para conectar a la interfaz OBDII a través de un cable DBG-OB
2. V_OBD: Se alimenta de la interfaz OBDII (desde DB9)
3. Indicador LED:
 - PWR: Potencia
 - TX: Parpadea cuando se envían datos
 - RX: Parpadea cuando se reciben datos
 - INT: Interrupción de datos
4. Terminal: CAN_H y CAN_L
5. Pines de Arduino UNO
6. Conector Serial Grove
7. Conector I2C Grove
8. Pines ICSP
9. CI - MCP2551, un transceptor CAN de alta velocidad
10. CI - MCP2515, controlador CAN independiente con interfaz SPI

2.4 Sistema Electrónico con Arduino MKR

2.4.1 Arduino MKR WiFi 1010

Esta placa es parte de la familia de placas MKR, cada una de las cuales cuenta con un módulo de radio que facilita la comunicación a través de Wi-Fi, Bluetooth, LoRa, Sigfox y NB-IoT. Todas las placas de esta serie se fundamentan en el procesador de bajo consumo Arm Cortex-M0 de 32 bits SAMD21, y además incluyen un chip criptográfico para asegurar la comunicación de manera confiable.

La sustitución de la placa Arduino Mega 2560 por una Arduino MKR WiFi 1010 permitirá la incorporación de comunicaciones inalámbricas con el vehículo robótico. Además, se aumenta la velocidad del reloj de 16 MHz a 48 MHz , lo que implica un incremento en la rapidez con la que la placa recupera e interpreta instrucciones.

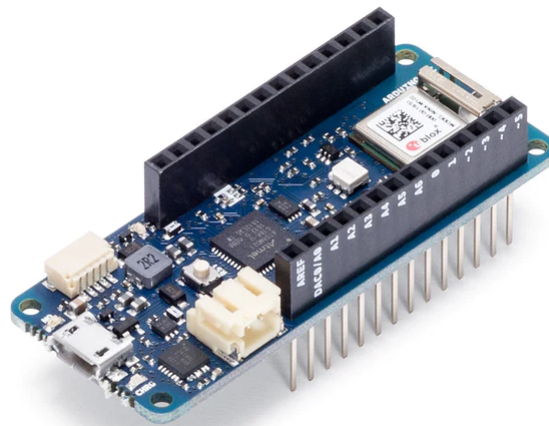


Figura 2.6: Arduino MKR WiFi 1010 `ArduinoMKRWiFi1010`

Sus especificaciones técnicas son:

Especificaciones técnicas	
Microcontrolador	MCU ARM de bajo consumo SAMD1 Cortex-M0+ de 32 bits
Módulo Radio	u-block NINA-W102
Alimentación	5V
Elemento Seguro	ATECC508
Batería Compatible	Li-Po Single Cell, 3.7V, 1024mAh Mínimo
Voltaje de Operación del Circuito	3,3V
Pines E/S Digital	8
Pines PWM	13
UART	1
SPI	1
I2C	1
Pines Entrada Analógica	7
Pines Salida Analógica	1
Interrupciones Externas	10
Corriente Continua por Pin E/S	7mA
Memoria Flash CPU	256KB
SRAM	32KB
EEPROM	no
Velocidad de Reloj	32,768kHz (RTC), 48MHz
LED Integrado	6
USB	Dispositivo USB de alta velocidad y host integrado
Largo	61,5mm
Ancho	25mm
Peso	32gr

Tabla 2.2: Especificaciones técnicas del Arduino MKR

2.4.2 Arduino MKR CAN Shield

La MKR CAN Shield simplifica la conexión de las placas MKR con sistemas industriales, especialmente con aplicaciones automovilísticas. Esta *shield* añade posibles aplicaciones en vehículos inteligentes, coches autónomos y drones, al facilitar la conexión a través del protocolo CAN. Además, permite conectar directamente una placa MKR a diversos tipos de sensores, motores y pantallas industriales, brindando una solución versátil para una amplia gama de proyectos.

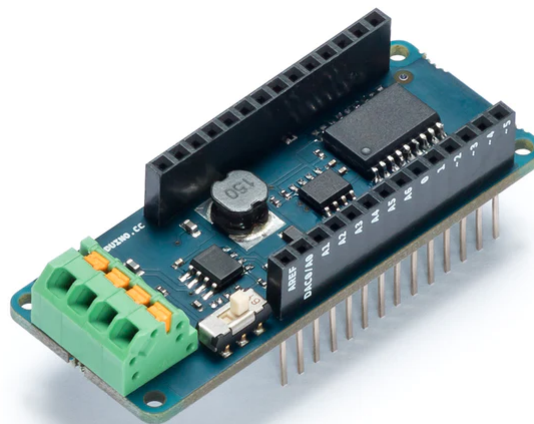


Figura 2.7: Arduino MKR CAN Shield **ArduinoMKRCANShield**

Sus especificaciones técnicas son:

Especificaciones técnicas	
Protocolo	CAN Bus
Interfaz	SPI
Voltaje de funcionamiento del circuito	3,3V
Controlador	Microchip MCP2515
Transceptor	NXP TJA1049
Convertidor Buck	Texas Instruments TPS54232
V _{in} ()	7V – 24V
V _{in} ()	5V
Compatibilidad	MKR

Tabla 2.3: Especificaciones técnicas del Arduino MKR Shield

2.5 Chip MCP2515

El MCP2515 del fabricante Microchip Technology es el controlador que encontramos en las *shields* de Arduino utilizadas en este proyecto. Es un controlador CAN de segunda generación capaz de transmitir y recibir tanto tramas de datos estándar como extendidos a una velocidad máxima de 1Mb/s . Este microchip se comunica con microcontroladores a través de una Interfaz Periférica Serial (SPI) estándar en la industria.

18-Lead PDIP/SOIC

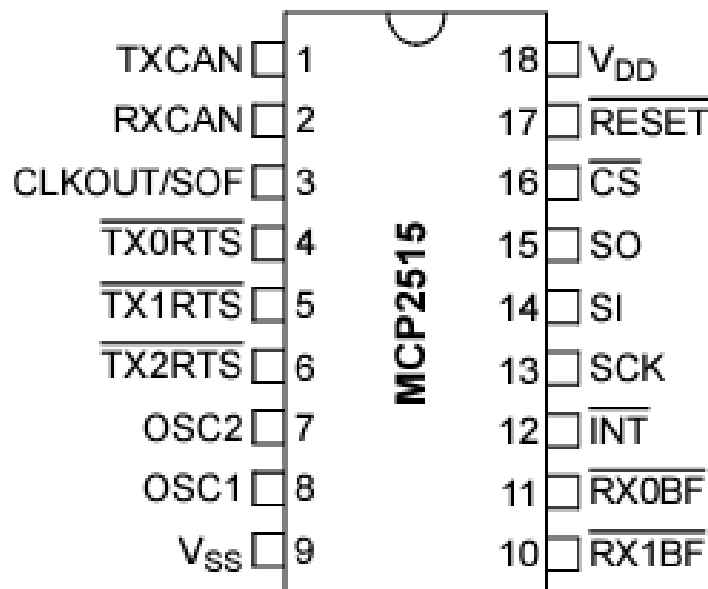


Figura 2.8: MCP2515 - Microchip Technology MCP2515

2.5.1 SPI - Serial Peripheral Interface

La interfaz de periféricos en serie (SPI) es un bus de interfaz utilizado habitualmente para enviar datos entre microcontroladores y pequeños periféricos como registros de desplazamiento y sensores. Incluye una línea de reloj, dato entrante, dato saliente y un pin de selección, que se encargará de elegir el dispositivo con el que se va a establecer la comunicación.

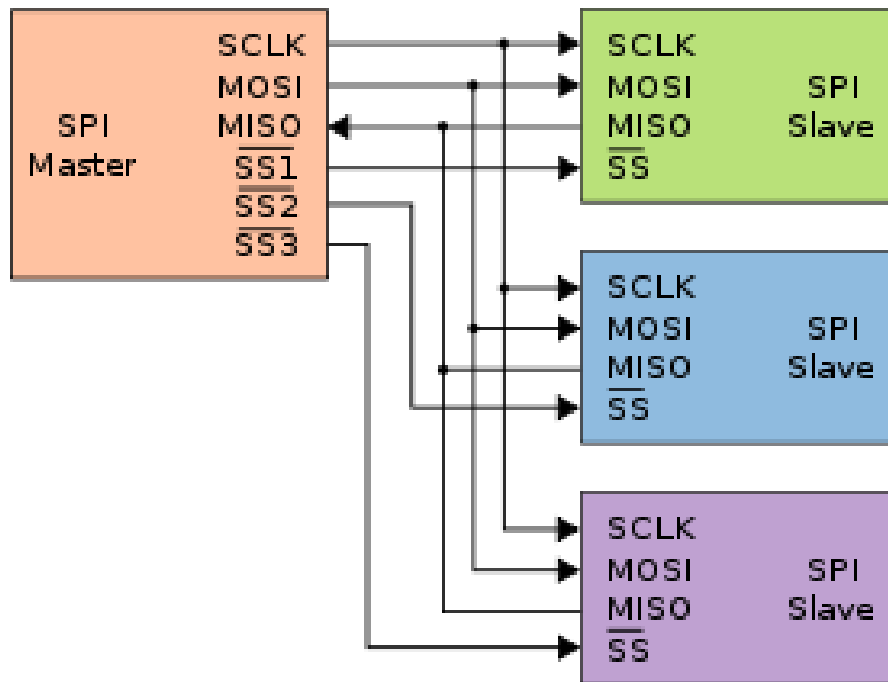


Figura 2.9: Conexión bus SPI Wiki-SPI

La principal diferencia entre este puerto serie y uno común se encuentra en la señal de reloj. Un puerto serie común, con líneas TX y RX, se conoce como “asíncrono” porque no hay control sobre cuándo se envían los datos ni ninguna garantía de que ambos lados estén funcionando a la misma velocidad. Dado que los ordenadores se basan normalmente en que todo está sincronizado con un único reloj, esto puede ser un problema cuando dos sistemas con relojes diferentes intentan comunicarse entre sí. SPI funciona con un bus de datos “síncrono”, lo que significa que utiliza líneas separadas para los datos y un reloj que mantiene ambos lados en perfecta sincronía, sin necesidad de especificar la velocidad, aunque si se deberá tener en cuenta las velocidades máximas a la que pueden funcionar los dispositivos.

2.5.2 CAN - Controller Area Network

El protocolo CAN Bus es un estándar de comunicación utilizado en sistemas electrónicos distribuidos. CAN es el acrónimo de *Controller Area Network*, es decir, Red de Área de Controlador. En el ámbito de la informática hace referencia a un elemento que transmite una elevada cantidad de información.

Este protocolo fue desarrollado en la década de 1980 por la compañía alemana Bosch, inicialmente para su uso en automóviles, pero hoy en día su uso se ha extendido a diferentes sectores, incluyendo el industrial, el médico y el de servicios públicos.

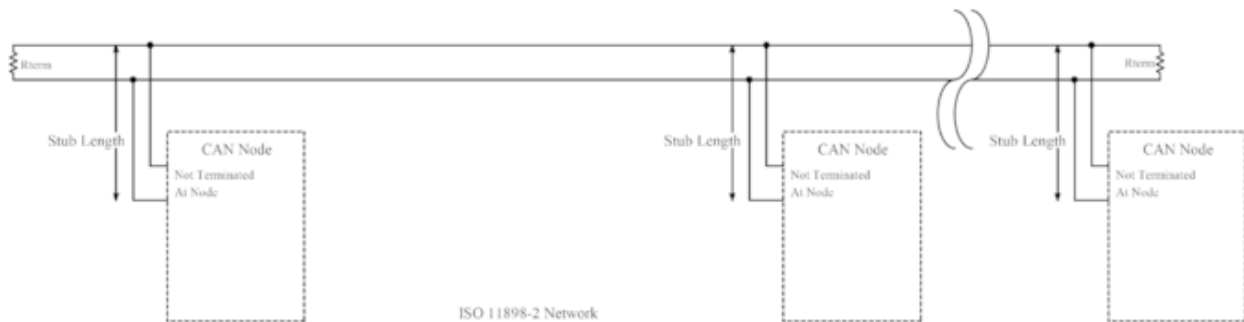


Figura 2.10: Conexión bus CAN Wiki-CAN

Los mensajes se transmiten mediante un bus de datos compartido, lo que significa que todos los dispositivos conectados a la red pueden recibir el mensaje. Cada mensaje tiene una identificación única que sirve para distinguirlo de otros mensajes en la red.

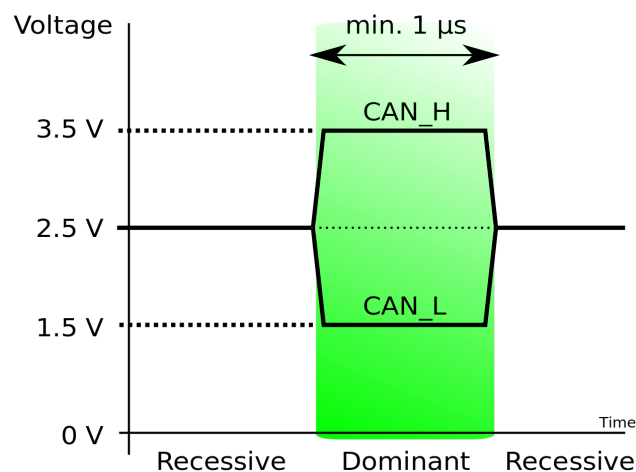


Figura 2.11: Niveles de tensión del CAN Bus Wiki-CAN

Para interconectar los distintos dispositivos solo son necesarios dos cables (CAN_H y CAN_L) y una resistencia de 120Ω . Dependiendo del voltaje de estas señales, el bus puede encontrarse en modo recesivo, donde ambos están al mismo nivel de tensión, o en modo dominante, en la que se encuentran con una diferencia de tensión mínima de $1,5V$. La lectura de los bits se basa en esta diferencia de voltaje entre los cables, lo que proporciona una mayor protección frente a interferencias electromagnéticas.

Capítulo 3

Software utilizado

3.1 Introducción

En este capítulo se recopilará el software empleado en el desarrollo del proyecto. Se procede a dividir el capítulo creando secciones para cada programa y exponiéndolos brevemente.

3.2 Arduino IDE

El entorno de desarrollo integrado (IDE) de Arduino es una aplicación multiplataforma (Linux, Windows y macOS) que permite escribir y cargar programas en placas Arduino. Admite lenguajes de programación como C y C++. El programa solo requiere de dos funciones básicas para funcionar, el *setup* y *loop*.



Figura 3.1: Logotipo Arduino. **Arduino**

En este proyecto se utiliza esta IDE para desarrollar códigos básicos que permitan una primera toma de contacto de la comunicación CAN Bus entre la placa y el motor.

3.3 TeraTerm

TeraTerm es un software gratuito que permite emular sistemas de comunicación.



Figura 3.2: Logotipo TeraTerm. **TeraTerm**

El motor GIM8108 V3 incluye un puerto UART el cual permite acceder al firmware del motor. Para realizar esta conexión es necesario la utilización de un convertidor USB a TTL UART, decidiéndose utilizar el DSD TECH SH-U09C (figura 3.3).



Figura 3.3: Convertidor de USB a TTL UART. **TTL**

Una vez realizado el conexionado físico se procede a la configuración del interfaz serie en el ordenador. Para ello se establece una nueva conexión serie en TeraTerm con el puerto de comunicación del convertidor USB. Dicha conexión deberá ser configurada a *921600 baud, 8 bits, 1 stop bit y no parity bits*.

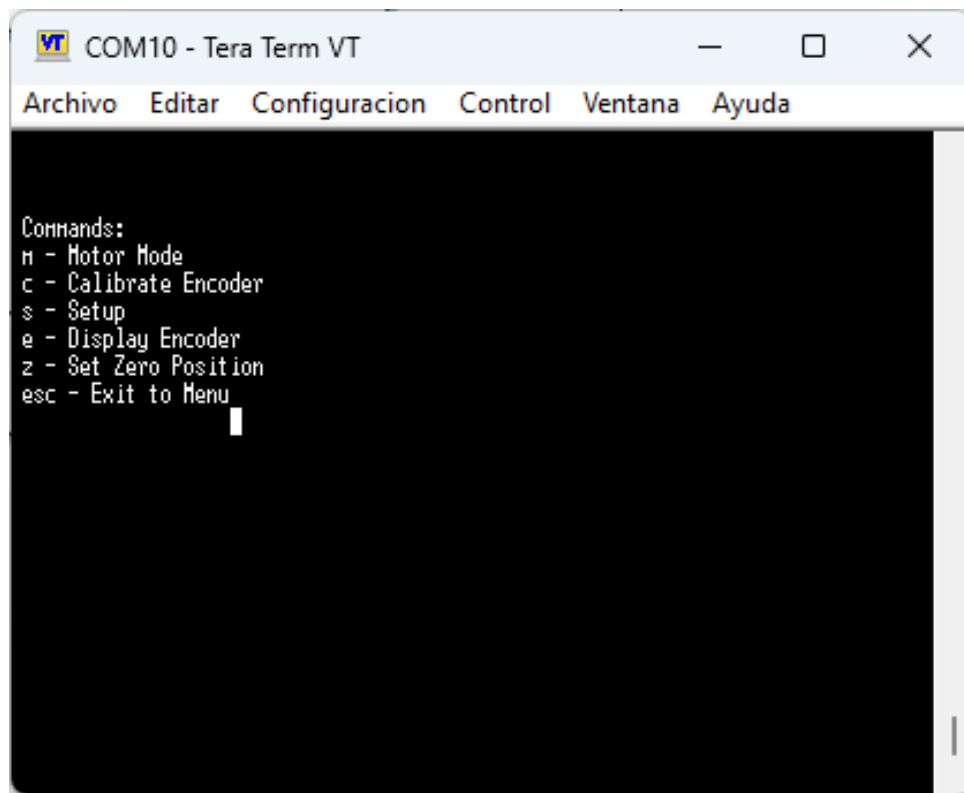


Figura 3.4: Menu (TeraTerm).

Este menú permite realizar las siguientes acciones sobre el motor:

- *Motor Mode*: En este modo el motor recibirá posición/velocidad/torque a través del CAN y enviará la posición real, velocidad y corriente.
- *Calibrate Encoder*: Este proceso realiza el calibrado del sensor de posición del rotor.
- *Setup*: Permite el ajuste de ciertos parámetros internos del motor (figura 3.5).
- *Display Encoder*: Muestra en tiempo real la lectura de los encoders (ángulo en radianes y cuenta).
- *Set Zero Position*: Sitúa a cero la posición actual del encoder.

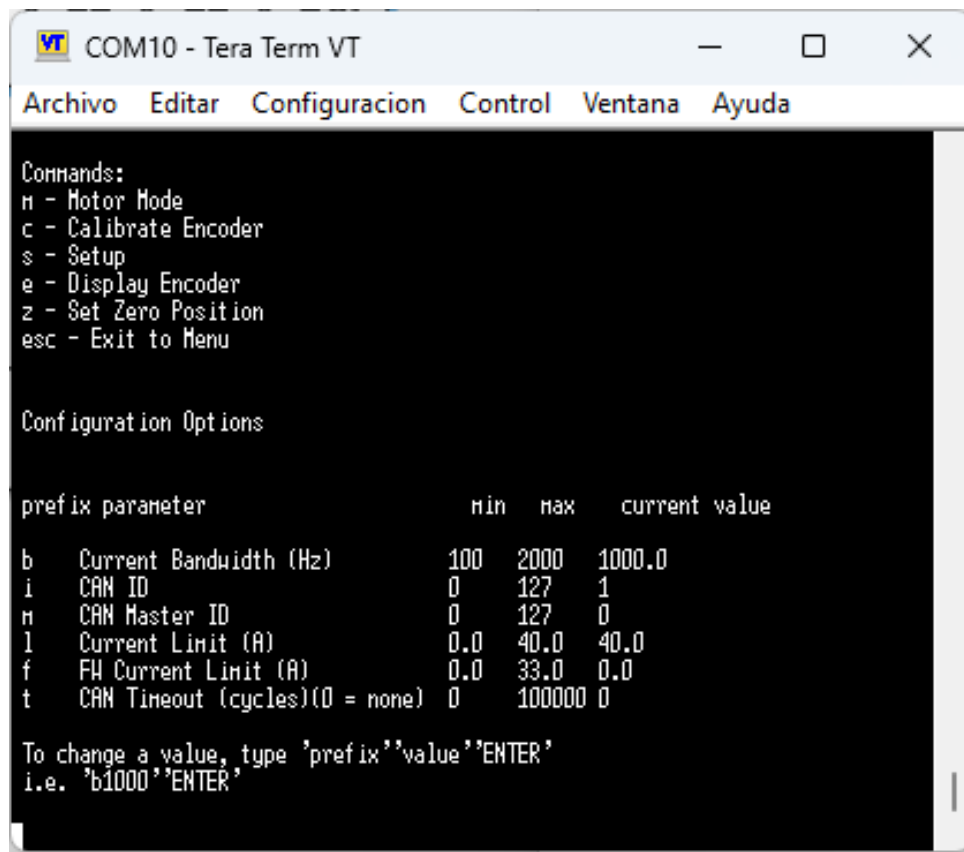


Figura 3.5: Setup (TeraTerm).

3.4 MATLAB

MATLAB es una plataforma de programación y cálculo numérico utilizada por millones de ingenieros y científicos para analizar datos, desarrollar algoritmos y crear modelos.



Figura 3.6: Logotipo MATLAB MATLAB.

Para el desarrollo de este proyecto, son necesarios una serie de paquetes específicos que se deben añadir mediante el apartado *Add-Ons* del propio MATLAB. Estos complementos son

los siguientes:

- MATLAB Support Package for Arduino Hardware
- Simulink Support Package for Arduino Hardware
- Embedded Coder
- MATLAB Coder
- ROS Toolbox
- Simulink Coder
- Vehicle Network Toolbox
- Simulink Real-Time

3.5 Simulink

Simulink es un entorno de diagramas de bloque que se utiliza para diseñar sistemas con modelos multidominio, simular antes de implementar en hardware y desplegar sin necesidad de escribir código.



Figura 3.7: Logotipo Simulink Simulink.

En este proyecto, Simulink es la herramienta más utilizada, puesto que es a partir de la cuál se crean los diversos modelos. Este, permite integrarlos en un solo proyecto, lo que facilita su exportación y organización.

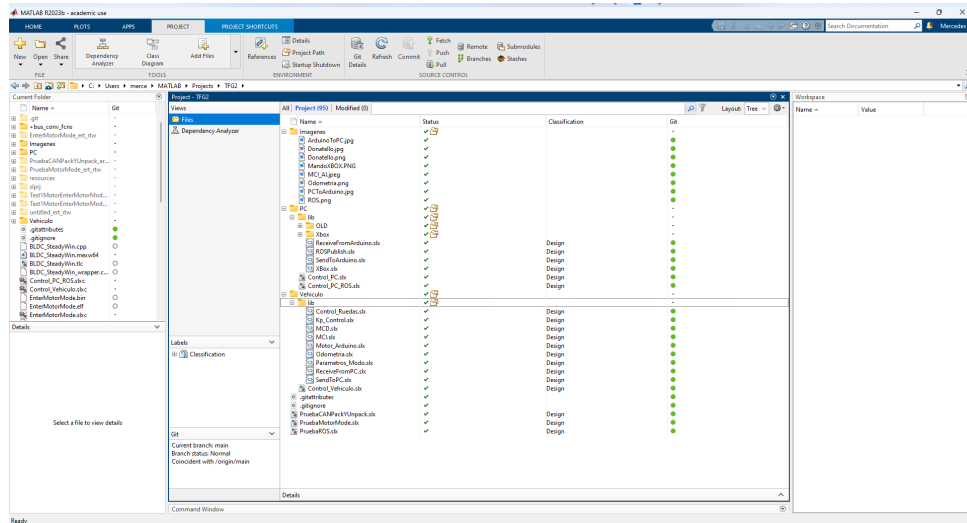


Figura 3.8: Contenido proyecto en Simulink

3.6 GitHub

GitHub es una plataforma de desarrollo colaborativo para alojar proyectos utilizando el sistema de control de versiones Git. Perteneciente a Microsoft desde el año 2018, se utiliza principalmente para la creación de código fuente de programas de ordenador. Además ofrece un servicio específico para estudiantes, conocido como Gitub Education, que permite a los estudiantes acceder a los beneficios de GitHub Pro.



Figura 3.9: Logotipo GitHub Wiki-GitHub

Se decide utilizar GitHub por su facilidad de enlace con Simulink. Esto permitirá mantener un mayor control sobre las modificaciones realizadas en el proyecto de Simulink, además de permitir la colaboración.

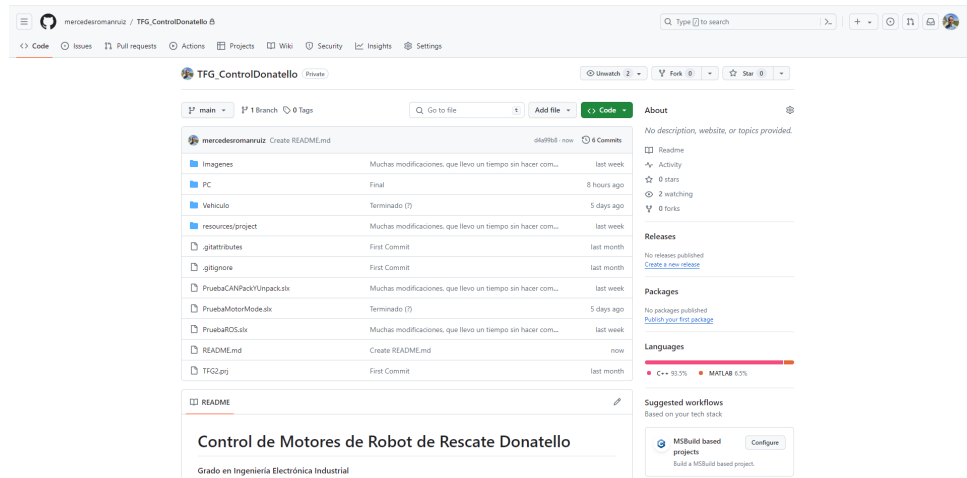


Figura 3.10: Reposito del proyecto en GitHub

3.7 ROS

Robot Operating System, también conocido como ROS, es un marco de trabajo flexible para el desarrollo de software de robótica. No es un sistema operativo en el sentido tradicional, sino una colección de herramientas, bibliotecas y convenciones diseñadas para simplificar la tarea de crear comportamientos complejos y robustos en robots.



Figura 3.11: Logotipo ROS ROS

ROS proporciona una infraestructura de comunicación entre procesos, que permite a los desarrolladores escribir software modular en forma de nodos que pueden intercambiar información mediante mensajes.

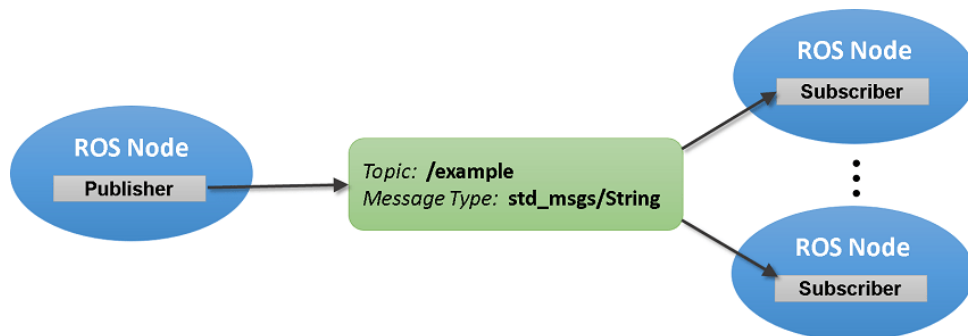


Figura 3.12: Ejemplo red ROS MathWorksROS

ROS2 es la segunda versión del framework de software ROS. Desarrollado por Open Robotics y una comunidad global de colaboradores, introduce mejoras significativas en términos de arquitectura, rendimiento y capacidades. En la siguiente tabla se elabora una comparativa entre ROS y ROS2:

Característica	ROS	ROS 2
Modelo de Comunicación	Maestro central, TCPROS	Distribuido, DDS
Plataformas Soportadas	Linux	Linux, Windows, macOS
Soporte en Tiempo Real	No está diseñado para tiempo real	Soporte nativo para tiempo real
Seguridad	Carece de mecanismos integrados	Autenticación, cifrado, autorización (basados en DDS)
Gestión de Nodos	Gestión manual	Composición dinámica y modularidad
Comunidad y Paquetes	Comunidad establecida, amplia base de paquetes	Desarrollo activo, foco en mejoras y nuevas características
Interfaz de Línea de Comandos (CLI)	ROS (catkin_make)	ROS 2 (colcon)
API de Cliente	rospy (Python), roscpp (C++)	rclpy (Python), rclcpp (C++)
Integración de Middleware	ROS Master y Slave (protocolo TCP/UDP)	DDS (Data Distribution Service)
Parámetros y Configuración	Parámetros gestionados por el nodo	Sistema de parámetros basado en DDS
Lanzamiento y Gestión de Procesos	Archivos de lanzamiento	Archivos de lanzamiento con sintaxis extendida y soporte para composiciones de nodos dinámicos
Compatibilidad con Versiones Anteriores	No compatible con ROS 2	No compatible con ROS

Tabla 3.1: Tabla Comparativa ROS y ROS 2.

Capítulo 4

Control de motores GIM8108v3

4.1 Introducción

En este capítulo se desarrollarán los distintos pasos seguidos en el desarrollo de este apartado del trabajo de fin de grado. Para ello, se empezó estudiando el firmware de los motores para determinar su funcionamiento y método de comunicación, con el fin de modelarlo posteriormente en Simulink

4.2 Firmware 1.9

Para entender como funciona el empaquetado y desempaquetado de mensajes CAN se utiliza el archivo proporcionado por el fabricante sobre el driver del motor **Katz2021**. En el archivo se enlaza un ejemplo de comunicación con el motor realizado por Ben Katz. A partir del código main.cpp se encuentra cómo se interpreta el mensaje CAN y cuáles son los parámetros que requiere el motor para funcionar. Véase el fragmento de código 4.1.

Código 4.1: main.cpp

```
1 /// Value Limits ///
```

```
2 #define P_MIN -12.5 f
```

```
3 #define P_MAX 12.5 f
```

```
4 #define V_MIN -45.0 f
```

```
5 #define V_MAX 45.0 f
```

```
6 #define KP_MIN 0.0 f
```

```
7 #define KP_MAX 500.0 f
```

```
8 #define KD_MIN 0.0 f
```

```
9 #define KD_MAX 5.0 f
```

```
10 #define T_MIN -18.0 f
```

```
11 #define T_MAX 18.0 f
```

```
12
```

```
13 /// CAN Command Packet Structure ///
```

```
14 /// 16 bit position command, between -4*pi and 4*pi
```

```
15 /// 12 bit velocity command, between -30 and + 30 rad/s
```

```
16 /// 12 bit kp, between 0 and 500 N-m/rad
17 /// 12 bit kd, between 0 and 100 N-m*s/rad
18 /// 12 bit feed forward torque, between -18 and 18 N-m
19 /// CAN Packet is 8 8-bit words
20 /// Formatted as follows. For each quantity, bit 0 is LSB
21 /// 0: [position[15-8]]
22 /// 1: [position[7-0]]
23 /// 2: [velocity[11-4]]
24 /// 3: [velocity[3-0], kp[11-8]]
25 /// 4: [kp[7-0]]
26 /// 5: [kd[11-4]]
27 /// 6: [kd[3-0], torque[11-8]]
28 /// 7: [torque[7-0]]
29
30 void pack_cmd(CANMessage * msg, float p_des, float v_des, float kp, float kd, float t_ff){
31     /// limit data to be within bounds ///
32     p_des = fminf(fmaxf(P_MIN, p_des), P_MAX);
33     v_des = fminf(fmaxf(V_MIN, v_des), V_MAX);
34     kp = fminf(fmaxf(KP_MIN, kp), KP_MAX);
35     kd = fminf(fmaxf(KD_MIN, kd), KD_MAX);
36     t_ff = fminf(fmaxf(T_MIN, t_ff), T_MAX);
37     /// convert floats to unsigned ints ///
38     int p_int = float_to_uint(p_des, P_MIN, P_MAX, 16);
39     int v_int = float_to_uint(v_des, V_MIN, V_MAX, 12);
40     int kp_int = float_to_uint(kp, KP_MIN, KP_MAX, 12);
41     int kd_int = float_to_uint(kd, KD_MIN, KD_MAX, 12);
42     int t_int = float_to_uint(t_ff, T_MIN, T_MAX, 12);
43     /// pack ints into the can buffer ///
44     msg->data[0] = p_int>>8;
45     msg->data[1] = p_int&0xFF;
46     msg->data[2] = v_int>>4;
47     msg->data[3] = ((v_int&0xF)<<4)|(kp_int>>8);
48     msg->data[4] = kp_int&0xFF;
49     msg->data[5] = kd_int>>4;
50     msg->data[6] = ((kd_int&0xF)<<4)|(t_int>>8);
51     msg->data[7] = t_int&0xff;
52 }
53
54 /// CAN Reply Packet Structure ///
55 /// 16 bit position, between -4*pi and 4*pi
56 /// 12 bit velocity, between -30 and + 30 rad/s
57 /// 12 bit current, between -40 and 40;
58 /// CAN Packet is 5 8-bit words
59 /// Formatted as follows. For each quantity, bit 0 is LSB
60 /// 0: [position[15-8]]
61 /// 1: [position[7-0]]
62 /// 2: [velocity[11-4]]
63 /// 3: [velocity[3-0], current[11-8]]
64 /// 4: [current[7-0]]
65
66 void unpack_reply(CANMessage msg){
67     /// unpack ints from can buffer ///
68     int id = msg.data[0];
69     int p_int = (msg.data[1]<<8)|msg.data[2];
70     int v_int = (msg.data[3]<<4)|(msg.data[4]>>4);
71     int i_int = ((msg.data[4]&0xF)<<8)|msg.data[5];
72     /// convert ints to floats ///
73     float p = uint_to_float(p_int, P_MIN, P_MAX, 16);
74     float v = uint_to_float(v_int, V_MIN, V_MAX, 12);
75     float i = uint_to_float(i_int, -I_MAX, I_MAX, 12);
76
77     if(id == 2){
78         theta1 = p;
79         dtheta1 = v;
80     }
81     else if(id == 3){
82         theta2 = p;
83         dtheta2 = v;
84     }
85 }
```

```
85 //printf("%d  %.3f  %.3f  %.3f\n\r", id, p, v, i);  
86 }  
87
```

En las primeras líneas se definen los valores máximos y mínimos de los parámetros del motor. En el caso del motor utilizado en este proyecto, estos parámetros varían ligeramente.

- Posición: En el firmware sus límites están en $-12,5 \text{ rad}$ y $12,5 \text{ rad}$ aunque los motores de SteadyWin funcionan entre -95 rad y 95 rad .
- Velocidad: Comprendida entre -45 rad/s y 45 rad/s .
- Kp: Con valores de 0 a $500 \text{ N} - \text{m/rad}$
- Kd: Sus límites están entre 0 y $5 \text{ N} - \text{m} \cdot \text{s/rad}$
- Torque: Comprendido entre $-18 \text{ N} \cdot \text{m}$ y $18 \text{ N} \cdot \text{m}$.

Los valores de estos parámetros condicionarán el controlador que ejecutará el motor. Diferenciando:

- Control de Posición: Se encarga de mantener la posición del motor. En este caso influye los parámetros Posición, Velocidad, Kp y Kd.
- Control de Velocidad: Este se encargará de mantener una velocidad constante. Los parámetros modificables en este caso son Kd y Velocidad.
- Control de Torque: No se utiliza en este proyecto. Cabe destacar que este tipo de control permite mantener una velocidad constante incluso cuando la carga varía.

Posteriormente, se declaran las funciones *unpack_reply* y *pack_cmd*, encargadas del empaquetado y desempaquetado de los mensajes CAN.

- *pack_cmd*: Encargada de empaquetar el mensaje CAN que se enviará al motor. 8 *bytes* de información que incluyen Posición, Velocidad, Kp, Kd y Torque.
- *unpack_reply*: Se encarga del desempaquetado del mensaje CAN recibido. Este tendrá un tamaño de 5 *bytes*, en los que se almacena Posición, Velocidad y Corriente.

4.3 Modelo Simulink

4.3.1 Bloques CAN

Haciendo uso del paquete *Vehicle Network Toolbox* de MATLAB y Simulink, se añaden al modelo los bloques *CAN Pack* y *CAN Unpack*.

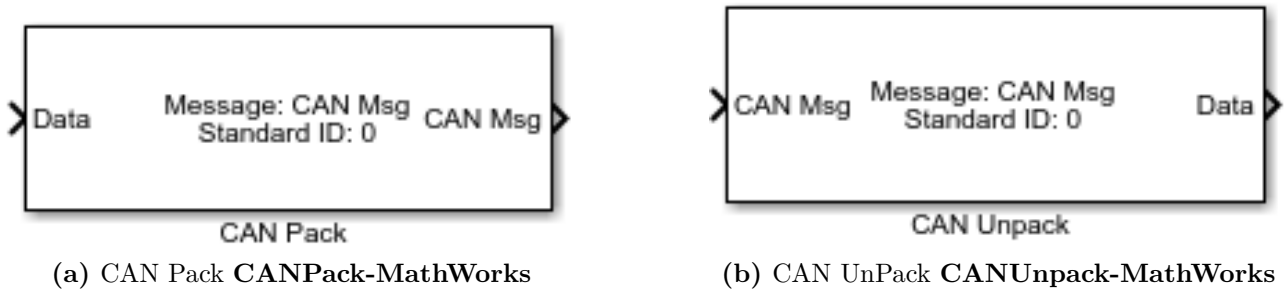


Figura 4.1: Bloques CAN

CAN Pack

El bloque CAN Pack carga datos de señal en un mensaje CAN a intervalos específicos durante la simulación. Para poder configurar dicho bloque seleccionamos que *Data input* sea *manually specified signals*. Esto nos permitirá crear las señales específicas de nuestro motor. Este bloque también permite la entrada de estas especificaciones a través de archivos del tipo CANdb o ARXML.

Block Parameters: CAN Pack

CAN Pack (mask) (link)

Pack data into a CAN message. Specify signals manually, or using definitions from DBC and ARXML files.

Parameters

Data is input as: manually specified signals

Database file: Browse...

Message list: (none)

Message

Name: CAN Msg

Identifier type: Standard (11-bit identifier)

CAN Identifier: 1

Length (bytes): 8

☐ Remote frame

☒ Output as bus

Add signal Delete signal

Signals

Name	Start bit	Length (bits)	Byte order	Data type	Multiplex type	Multiplex value	Factor	Offset	Min	Max
Position	8	16	BE	signed	Standard	0	0.0028992...	-95	-95	95
Velocity	28	12	BE	signed	Standard	0	0.0219780...	-45	-45	45
Kp	32	12	BE	unsigned	Standard	0	0.1221001...	0	0	500
Kd	52	12	BE	unsigned	Standard	0	0.0012210...	0	0	5
Torque	56	12	BE	signed	Standard	0	0.0087912...	-18	-18	18

OK Cancel Help Apply

Figura 4.2: CAN Pack - Block Parameters

A partir de la información obtenida analizando el código 4.1, se concluye que *Byte order* es *BE (Big-Endian)* y *Data type* es *unsigned*. Los valores máximos y mínimos introducidos en esta tabla son los valores físicos. *Factor* y *Offset* se deben calcular para cada una de las señales. Utilizando la expresión proporcionada por MATLAB **CANPack-MathWorks** :

$$raw_value = (physical_value - Offset) / Factor \quad (4.1)$$

Sabiendo que el valor mínimo de la señal corresponde a un valor *raw* de '0' y el máximo a ' $2^{bits} - 1$ ', se calculan los *Factor* y *Offset* para cada una de estas señales.

Código 4.2: Código de cálculo de Factor y Offset (CAN Pack) en MATLAB

```
1 % raw_value = (physical_value - Offset) / Factor
2
3 % ----- Position -----
4 syms Offset Factor;
5
6 Min = -95;
7 Max = 95;
8 bits = 16;
9
10 [Factor, Offset] = solve(0 == (Min - Offset) / Factor, 2^bits - 1 == (Max - Offset) / Factor, ...
    Factor, Offset);
11 Factor = vpa(Factor) % Aproximación de Factor
12 Offset
13
14 % ----- Velocity -----
15 syms Offset Factor;
16
17 Min = -45;
18 Max = 45;
19 bits = 12;
20
21 [Factor, Offset] = solve(0 == (Min - Offset) / Factor, 2^bits - 1 == (Max - Offset) / Factor, ...
    Factor, Offset);
22 Factor = vpa(Factor) % Aproximación de Factor
23 Offset
24
25 % ----- Kp -----
26 syms Offset Factor;
27
28 Min = 0;
29 Max = 500;
30 bits = 12;
31
32 [Factor, Offset] = solve(0 == (Min - Offset) / Factor, 2^bits - 1 == (Max - Offset) / Factor, ...
    Factor, Offset);
33 Factor = vpa(Factor) % Aproximación de Factor
34 Offset
35
36 % ----- Kd -----
37 syms Offset Factor;
38
39 Min = 0;
40 Max = 5;
41 bits = 12;
42
43 [Factor, Offset] = solve(0 == (Min - Offset) / Factor, 2^bits - 1 == (Max - Offset) / Factor, ...
    Factor, Offset);
44 Factor = vpa(Factor) % Aproximación de Factor
45 Offset
46
47 % ----- Torque -----
```

```

48 syms Offset Factor;
49
50 Min = -18;
51 Max = 18;
52 bits = 12;
53
54 [Factor, Offset] = solve(0 == (Min - Offset) / Factor, 2^bits - 1 == (Max - Offset) / Factor, ...
    Factor, Offset);
55 Factor = vpa(Factor) % Aproximación de Factor
56 Offset

```

Por último, sabiendo el orden que siguen los bits y su longitud, obtenemos la posición del bit de inicio de cada una.

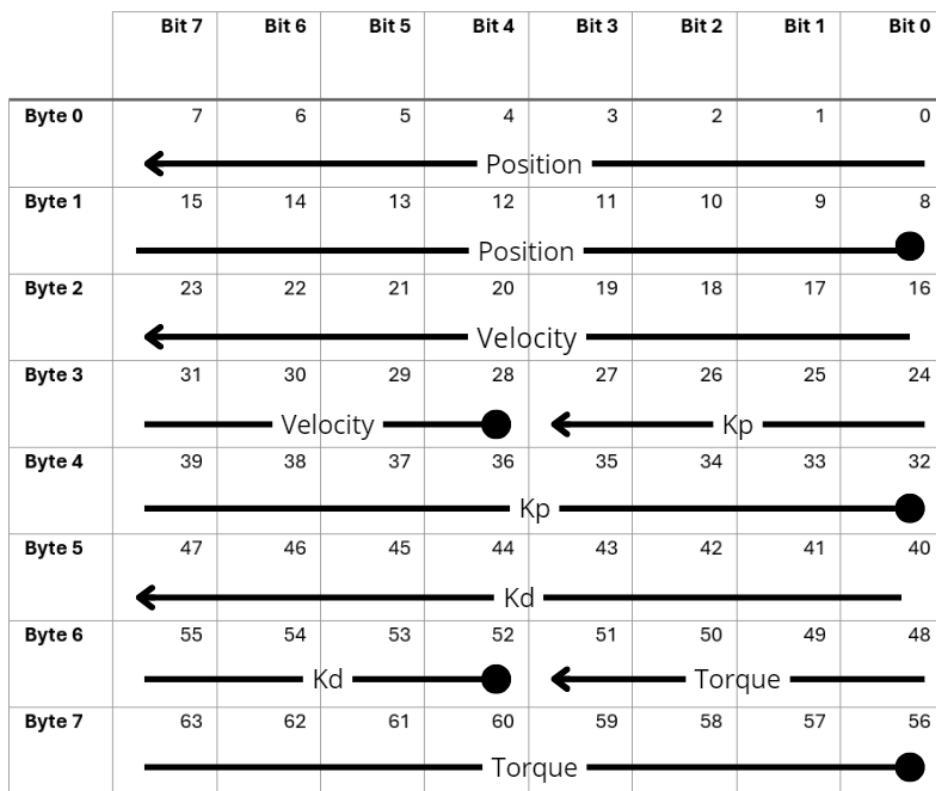
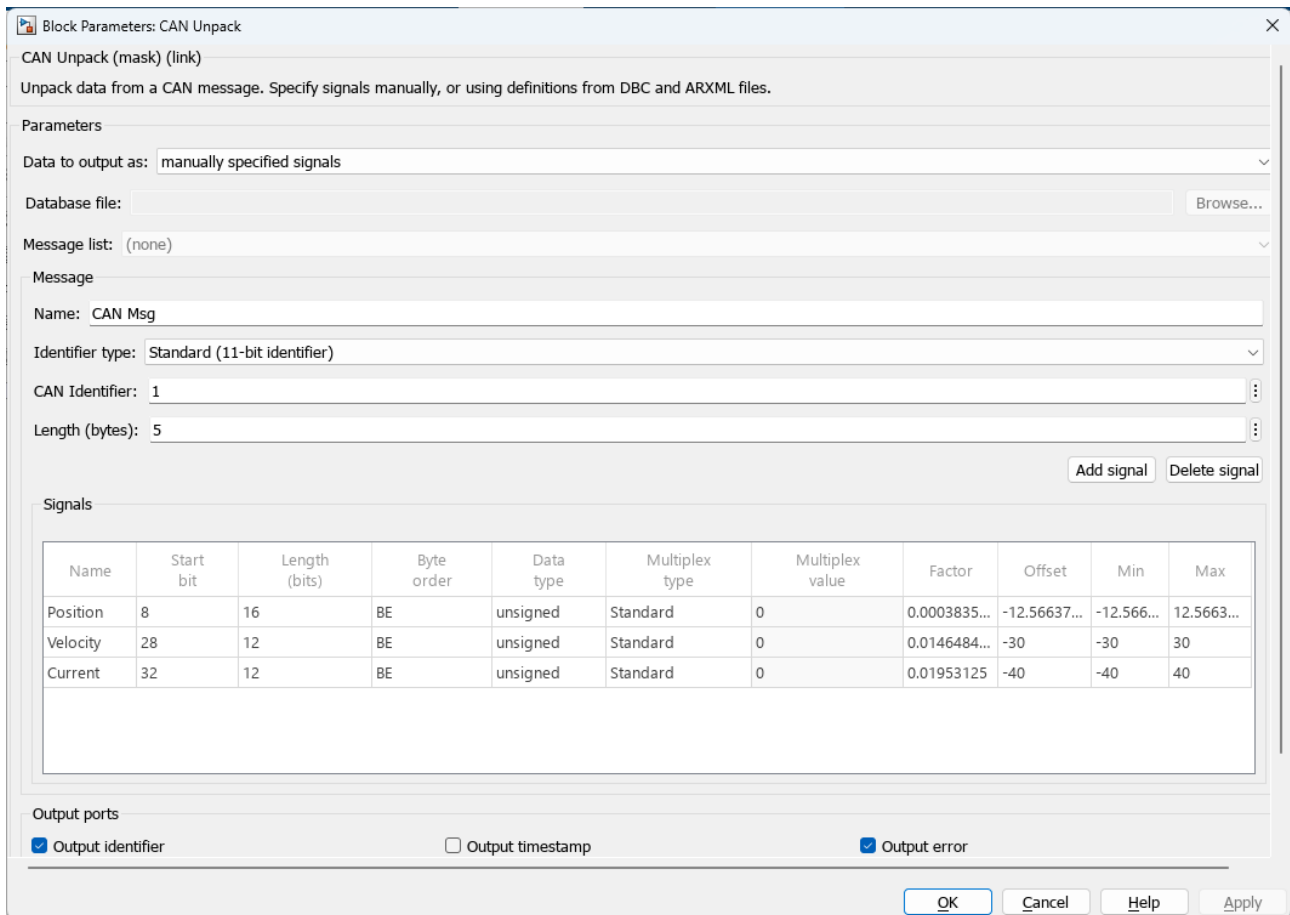


Figura 4.3: Estructura del mensaje CAN enviado al motor.

CAN Unpack

El bloque CAN Unpack descomprime un mensaje CAN en datos de señal utilizando los parámetros de salida especificados. Estos datos se emiten como señales individuales. Para poder configurar dicho bloque seleccionamos que *Data input* sea *manually specified signals*. Esto nos va a permitir crear las señales específicas de nuestro motor. Este bloque, al igual que el de CAN Pack, permite la entrada de estas especificaciones a través de archivos del tipo CANDb o ARXML.



Block Parameters: CAN Unpack

CAN Unpack (mask) (link)

Unpack data from a CAN message. Specify signals manually, or using definitions from DBC and ARXML files.

Parameters

Data to output as: manually specified signals

Database file: Browse...

Message list: (none)

Message

Name: CAN Msg

Identifier type: Standard (11-bit identifier)

CAN Identifier: 1

Length (bytes): 5

Add signal Delete signal

Signals

Name	Start bit	Length (bits)	Byte order	Data type	Multiplex type	Multiplex value	Factor	Offset	Min	Max
Position	8	16	BE	unsigned	Standard	0	0.0003835...	-12.56637...	-12.566...	12.5663...
Velocity	28	12	BE	unsigned	Standard	0	0.0146484...	-30	-30	30
Current	32	12	BE	unsigned	Standard	0	0.01953125	-40	-40	40

Output ports

☒ Output identifier ☐ Output timestamp ☒ Output error

OK Cancel Help Apply

Figura 4.4: CAN Unpack - Block Parameters

Observamos como el tamaño del mensaje enviado (5 *bytes*) y recibido (8 *bytes*) por el motor son distintos. El *Byte order* sigue siendo del tipo *BE* y *Data type unsigned*. Los valores máximos y mínimos que se introducen son los valores físicos. *Factor* y *Offset* se deben calcular nuevamente para cada una de las señales. Utilizando la expresión proporcionada por MATLAB **CANUnpack-MathWorks** :

$$physical_value = raw_value \times Factor + Offset \quad (4.2)$$

Sabiendo que el valor mínimo de la señal corresponde a un valor *raw* de '0' y el máximo a ' $2^{bits} - 1$ ', se calculan los *Factor* y *Offset* para cada una de estas señales.

Código 4.3: Código de cálculo de Factor y Offset (CAN UnPack) en MATLAB

```

1 % physical_value = raw_value * Factor + Offset
2
3 % ----- Position -----
4 syms Offset Factor;
5
6 Min = -4*pi;
7 Max = 4*pi;
8 bits = 16;

```

```

9
10 [Factor, Offset] = solve(Min == 0 * Factor + Offset, Max == (2^bits - 1) * Factor + Offset, ...
    Factor, Offset);
11 Factor = vpa(Factor) % Aproximación de Factor
12 Offset = vpa(Offset) % Aproximación de Offset
13
14 % ----- Velocity -----
15 syms Offset Factor;
16
17 Min = -30;
18 Max = 30;
19 bits = 12;
20
21 [Factor, Offset] = solve(Min == 0 * Factor + Offset, Max == (2^bits - 1) * Factor + Offset, ...
    Factor, Offset);
22 Factor = vpa(Factor) % Aproximación de Factor
23 Offset
24
25 % ----- Current -----
26 syms Offset Factor;
27
28 Min = -40;
29 Max = 40;
30 bits = 12;
31
32 [Factor, Offset] = solve(Min == 0 * Factor + Offset, Max == (2^bits - 1) * Factor + Offset, ...
    Factor, Offset);
33 Factor = vpa(Factor) % Aproximación de Factor
34 Offset

```

Por último, sabiendo el orden que siguen los bits y su longitud, obtenemos la posición del bit de inicio de cada una.

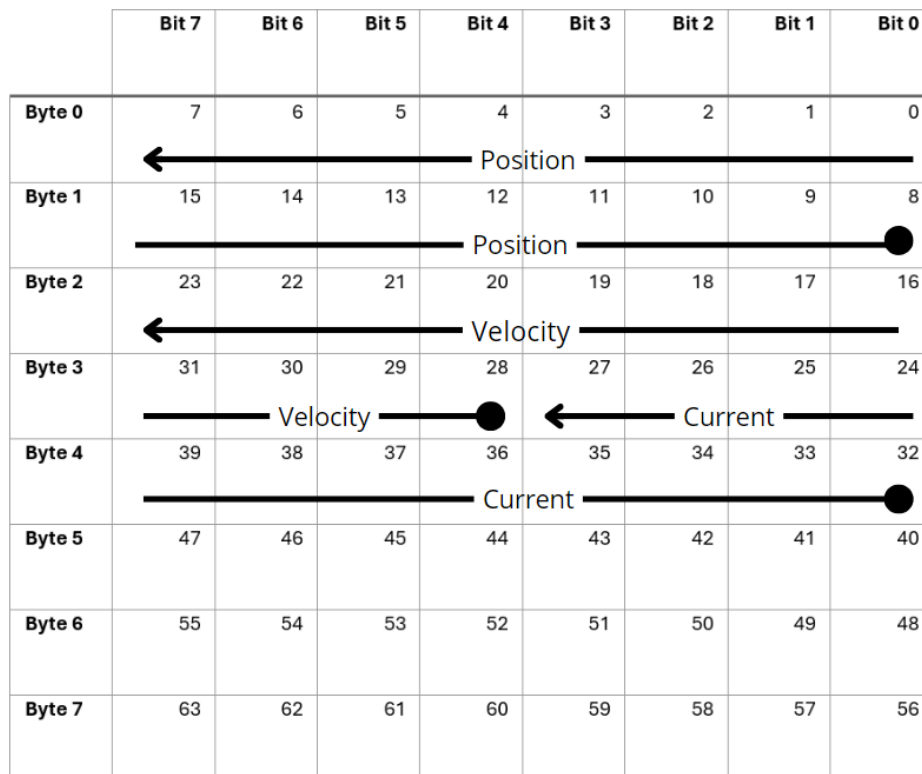


Figura 4.5: Estructura del mensaje CAN recibido desde el motor.

4.3.2 Subsistema comunicación motores

Los bloques configurados previamente serán aquellos que conformen la parte principal del subsistema ‘motores’. Este subsistema perteneciente al control del vehículo recibirá como entradas seis vectores, cada uno correspondiente a una rueda del robot, y una entrada del tipo *int* con la que se controlará el modo de funcionamiento de estas.

Cada uno de los vectores estará conformado por Posición, Velocidad, K_p , K_d y Par de Torsión. Asimismo, se establecen tres modos de operación de los motores:

- **Modo 0: Apagados** - En este modo los motores se encuentran apagados y por tanto no se tiene ningún tipo de control sobre los mismo.
- **Modo 1: Encendidos** - Este modo se mantendrá siempre y cuando los motores se encuentren encendidos y con una velocidad distinta de 0. Se empleará un control de velocidad cuyos parámetros modificables serán *Velocity* y K_d , permaneciendo los demás a 0.
- **Modo 2: Actualizar la posición cero** - Este modo de funcionamiento se activará de forma automática cuando el motor esté encendido y la velocidad de entrada sea 0. En este modo se pasará de un control de velocidad a uno de posición, de este modo se consigue que el vehículo se quede inmóvil independientemente del grado de inclinación de la superficie en la que se encuentre.

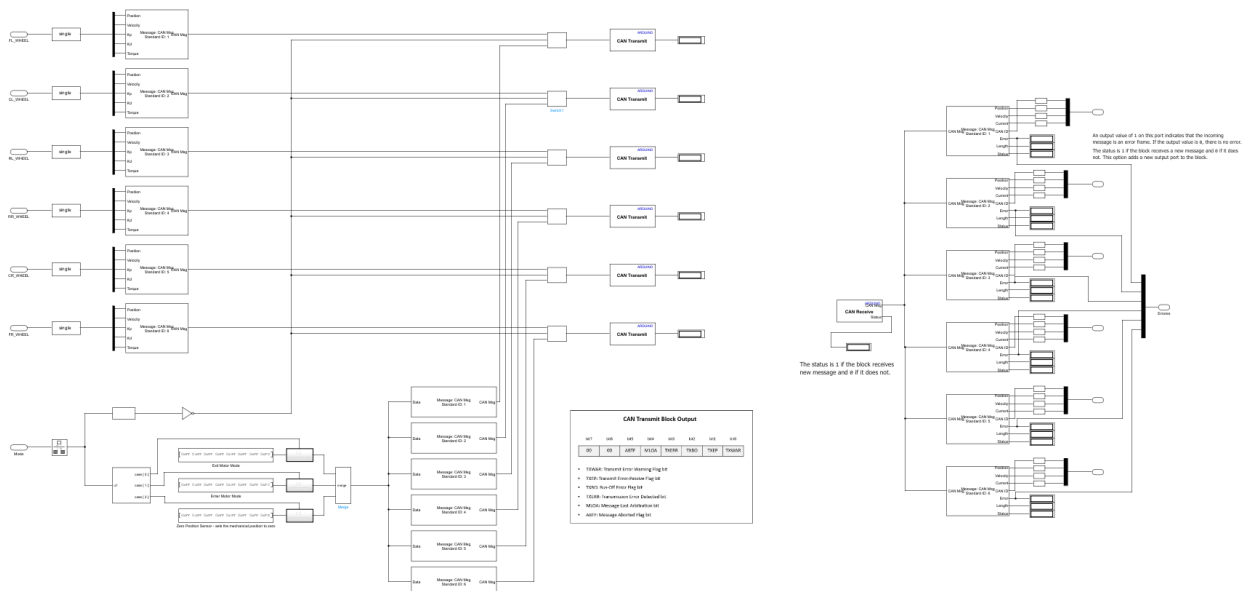


Figura 4.6: Contenido del subsistema Motores.

En este subsistema (Figura 4.6) se distinguen dos partes: transmisión a de información hacia los motores (Figura 4.7) y recepción de datos desde estos.

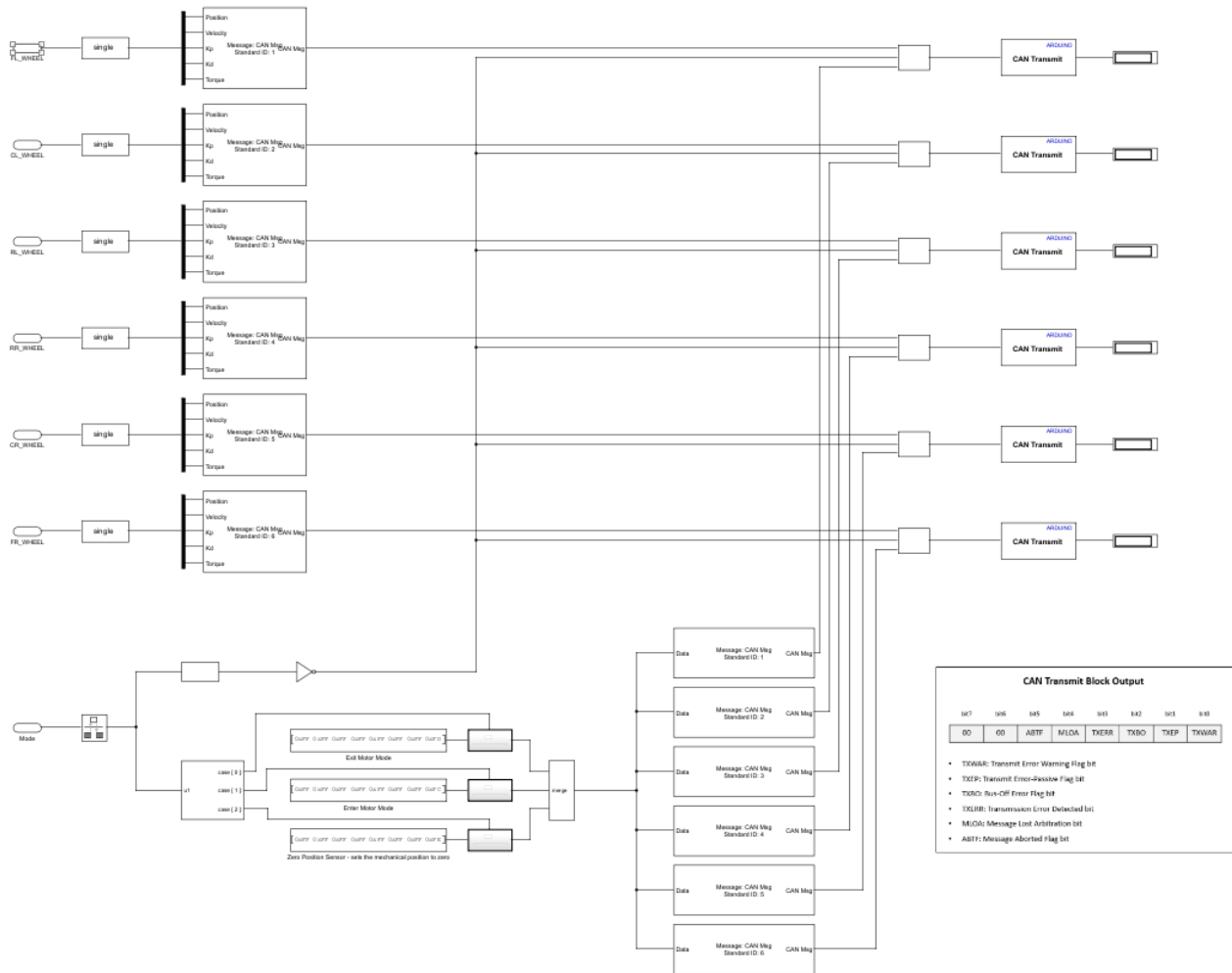


Figura 4.7: Transmisión de datos a los motores.

Esta primera parte recibe la entrada de ‘*Mode*’, que pasará por el bloque ‘*Rate Transition*’ (Figura 4.8a) antes de utilizarse. Este bloque intermediario se encargará de manejar la transferencia de datos entre bloques que operan a diferentes velocidades. Se procederá a detectar los cambios de modo haciendo uso del bloque ‘*Detect Change*’ (Figura 4.8b). Este determinará si la señal de entrada que está recibiendo no es igual a la anterior. Teniendo que establecerse una condición inicial, que en este caso, será ‘0’ puesto que los motores se encontraran inicialmente apagados. La salida de este bloque será un booleano con valor *TRUE* si detecta cambio y *FALSE* si no.



Figura 4.8: Bloques utilizados.

La señal ‘Mode’ también será recibida por el bloque ‘Switch Case’ que detectará el modo de funcionamiento encauzando así la señal que queremos que se transmita a los motores. Estos comandos especiales se encuentran en el documento de Ben Katz **Katz2021** y son:

- *Enter Motor Mode*: [0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFC]
- *Exit Motor Mode*: [0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFD]
- *Zero Position Sensor*: [0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFE]

El bloque ‘Merge’ se encargará de combinar las entradas en una única salida que será la entrada a los bloques ‘CAN Pack’ esta vez configurados seleccionando que ‘Data is input as’ es *raw data*.

En esta parte también se realiza el empaquetado de los parámetros de control de cada motor. Se utilizan las entradas correspondientes a los vectores de cada motor, estos se pasan inicialmente por un bloque de ‘Data Type Conversion’ para asegurar que todos los componentes del vector son del tipo *single*. Una vez hecho esto se sacan cada uno de los parámetros mediante el bloque ‘Demux’ y se conecta con el ‘CAN Pack’ con la configuración mostrada previamente en este capítulo de la memoria (Figura 4.2)

El valor negado proveniente del bloque ‘Detect Change’ será el que se utilice en los bloques ‘Switch’ para que la información que transmita finalmente a los motores sea el nuevo modo de funcionamiento o el control de estos. Se decide configurar que el bloque ‘CAN Transmit’ muestre su *status* para poder visualizar durante las pruebas si se está produciendo un error en la transmisión de datos. Añadiéndose junto a este, una captura de pantalla de la ayuda por parte de MathWorks para traducir este código de error fácilmente.

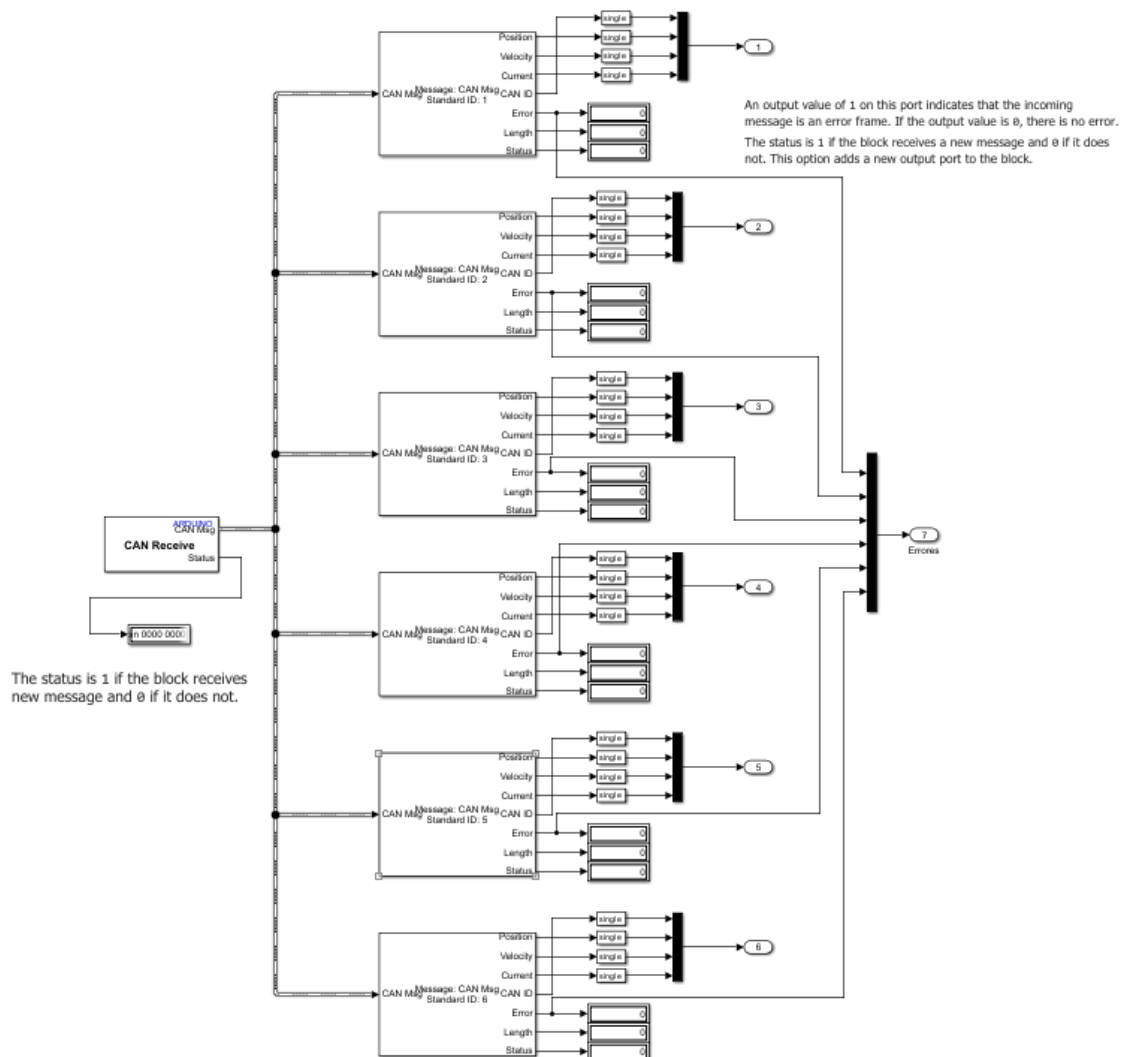


Figura 4.9: Recepción de datos de los motores.

Esta última parte es la encargada del desempaquetado de los mensajes CAN recibidos por el Arduino. El bloque ‘*CAN Receive*’ al igual que el ‘*CAN Transmit*’ es configurado para que muestre su *status*, esta vez para identificar si se está recibiendo un mensaje nuevo o no. Este bus se hace llegar a los diferentes bloques de de ‘*CAN Unpack*’ que se encuentran con la configuración mostrada previamente en la figura 4.4.

Los parámetros recibidos se multiplexaran en un vector para cada motor y pasarán a la salida del subsistema. Estos bloques ‘*CAN Unpack*’ además mostrarán señales de error, longitud y estado. Que servirán para determinar si el mensaje se trata de una trama de error o no, y si se está recibiendo nueva información o no.

4.4 Arduino MKR

Previamente en esta memoria se propuso la utilización de una placa Arduino MKR y su respectiva *shield*, que permitiese una posterior comunicación inalámbrica con el vehículo robótico. Se realiza por tanto un código básico (Anexo A) para probar la comunicación entre esta nueva placa y el motor. A continuación se describen las partes de interés del código.

Código 4.4: Definiciones Básicas

```
1 unsigned char Enter_Motor_Mode[8] = {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFC};
2 unsigned char Exit_Motor_Mode[8] = {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFD};
3
4 unsigned char buf;
5 unsigned char len = 0;
6 long unsigned int canID = 0;
7 unsigned char ID_type = 0;
```

En esta primera parte del código se inicializan las variables que vamos a utilizar posteriormente. *Enter_Motor_Mode* y *Exit_Motor_Mode* son las señales específicas del motor que hace que se enciendan y apaguen respectivamente. **Katz2021**

Código 4.5: Función Setup

```
1 void setup() {
2     // put your setup code here, to run once:
3     SERIAL.begin(115200);
4
5     while(CAN_OK != CAN.begin(0, CAN_1000KBPS, MCP_16MHZ)) {
6         SERIAL.println("CAN BUS Shield init fail!");
7         SERIAL.println("Init CAN BUS Shield again");
8         delay(10);
9     }
10
11     SERIAL.println("CAN BUS Shield init ok!");
12
13     CAN.sendMsgBuf(0x05, 0, 8, Enter_Motor_Mode);
14
15     delay(1000);
16
17     // CAN.sendMsgBuf(0x05, 0, 8, Exit_Motor_Mode);
18
19     // delay(1000);
20
21 }
```

En esta parte se recoge la configuración y se ejecutará una única vez. La función se encargará de inicializar la *CAN Bus Shield* y de informarnos a través del monitor serie si se ha realizado correctamente o si ha fallado. Además se envía la señal de encendido de motores una vez

Código 4.6: Función Loop

```
1 void loop() {
2     // put your main code here, to run repeatedly:
3
4 }
```

```
5
6  if(CAN_MSGAVAIL == CAN.checkReceive()){
7      CAN.readMsgBuf(&canID, &ID_type, &len, &buf);
8      SERIAL.println(len);
9
10     SERIAL.println("-----");
11     SERIAL.println("Get data from ID: 0x");
12     SERIAL.println(canID, HEX);
13
14     SERIAL.println(buf, HEX);
15     SERIAL.println("\t");
16
17     SERIAL.println();
18 }
19
20 else
21     SERIAL.println("No llega nada");
22
23 delay(1000);
24
25 }
```

Esta parte del código será la que se ejecute cíclicamente. En ella se comprueba la recepción de información a través del CAN. En caso de recibirse, imprime en el monitor serie la ID del motor del cuál se ha recibido el mensaje, y el mensaje cuestión. Todo esto se mostrará en formato hexadecimal. En caso de no recibir nada, se imprimirá en el monitor serie el mensaje ‘No llega nada’ que nos permitirá identificar que algo está fallando en la recepción de información.

Tras varios intentos se detecta que a pesar de la correcta inicialización de la *shield* de Arduino. La comunicación entre esta y el motor falla, puesto que al mandar la señal de encendido (*Enter_Motor_Mode*) estos no lo hacen. Por tanto se decide seguir con la placa Arduino Mega.

Capítulo 5

Introducción a ROS

5.1 Introducción

Durante el desarrollo de este proyecto se planteó la posibilidad implementar ROS en el controlador. Para ello fue necesario familiarizarse con el funcionamiento de este *middleware* robótico.

5.2 Software Adicional

5.2.1 VMware Workstation 17 Player

VMware Workstation 17 Player es una aplicación de virtualización de escritorio desarrollada por VMware. Permite a los usuarios ejecutar múltiples sistemas operativos simultáneamente en una sola máquina física mediante la creación y gestión de máquinas virtuales.



Figura 5.1: Logotipo VMware VMware

Esta aplicación será la que se utilice, con el sistema operativo Ubuntu, para realizar las simulaciones de pruebas con ROS.

Ubuntu

Ubuntu es una distribución de Linux basada en Debian, desarrollada y mantenida por Canonical Ltd. Es conocida por su enfoque en la facilidad de uso, accesibilidad y regularidad en las actualizaciones lo que la convierte en una opción popular tanto para usuarios principiantes como para profesionales. Una de las características destacadas de Ubuntu es su compromiso con el software libre y de código abierto, permitiendo a los usuarios modificar y distribuir el sistema operativo según sus necesidades.



Figura 5.2: Logotipo Ubuntu Ubuntu

Durante esta parte del proyecto, utilizaremos la versión de Ubuntu 20.04.6 LTS, proporcionada por MathWorks **MatlabROSVirtual**.

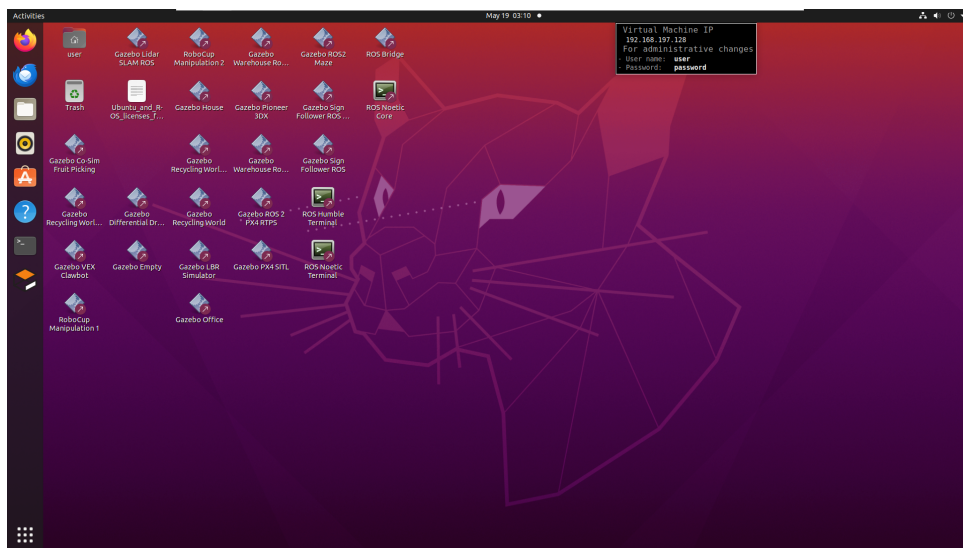


Figura 5.3: Escritorio de la máquina virtual `ros_noetic_foxy_gazebov11.vmx`

Gazebo

Gazebo es un simulador de robots en 3D de código abierto ampliamente utilizado en la investigación y desarrollo de robótica. Permite la simulación de sistemas robóticos complejos en un entorno visual realista, ofreciendo física avanzada, gráficos de alta calidad y una interfaz

para interactuar con los robots. Gazebo es capaz de simular diversos sensores, actuadores y entornos, lo que facilita la prueba y validación de algoritmos y diseños de robots sin necesidad de hardware.



Figura 5.4: Logotipo Gazebo **Gazebo**

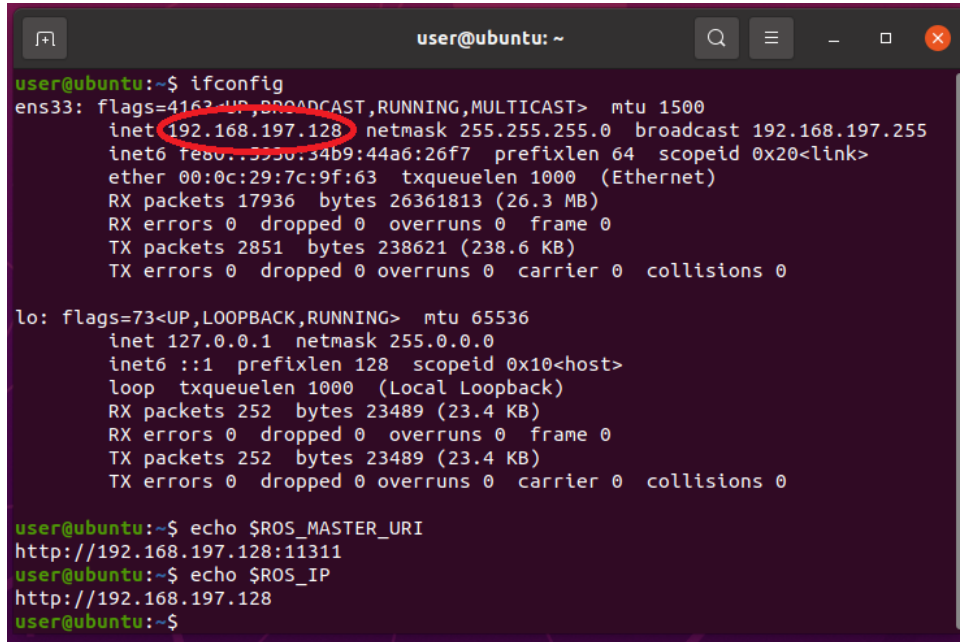
Este simulador permite su integración con ROS, es por ello por lo que se utiliza en esta parte del proyecto.

5.3 Configuración

Lo primero sería configurar la red, para ello se deben modificar dos variables del entorno ROS: `ROS_MASTER_IP` y `ROS_IP`. Estas variables se suelen establecer automáticamente, pero se deben comprobar. Para ello se abre la terminal de Ubuntu (Ctrl+Alt+T) y se escriben los siguientes comandos:

Código 5.1: Comprobación variables

```
1 ifconfig
2 echo $ROS_MASTER_URI
3 echo $ROS_IP
```



```
user@ubuntu: ~  
user@ubuntu:~$ ifconfig  
ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500  
    inet 192.168.197.128 netmask 255.255.255.0 broadcast 192.168.197.255  
    inet6 fe80::5550:34b9:44a6:26f7 prefixlen 64 scopeid 0x20<link>  
    ether 00:0c:29:7c:9f:63 txqueuelen 1000 (Ethernet)  
    RX packets 17936 bytes 26361813 (26.3 MB)  
    RX errors 0 dropped 0 overruns 0 frame 0  
    TX packets 2851 bytes 238621 (238.6 KB)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
  
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536  
    inet 127.0.0.1 netmask 255.0.0.0  
    inet6 ::1 prefixlen 128 scopeid 0x10<host>  
    loop txqueuelen 1000 (Local Loopback)  
    RX packets 252 bytes 23489 (23.4 KB)  
    RX errors 0 dropped 0 overruns 0 frame 0  
    TX packets 252 bytes 23489 (23.4 KB)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
  
user@ubuntu:~$ echo $ROS_MASTER_URI  
http://192.168.197.128:11311  
user@ubuntu:~$ echo $ROS_IP  
http://192.168.197.128  
user@ubuntu:~$
```

Figura 5.5: Configuración red.

En caso de que estas IPs no coincidan, se deberán cambiar usando los siguientes comandos, sustituyendo ‘IP_OF_VM’ con la IP que corresponde a la máquina virtual (Figura 5.5):

Código 5.2: Configuración variables

```
1 echo export ROS_MASTER_URI=http://IP_OF_VM:11311 >> ~/.bashrc  
2 echo export ROS_IP=IP_OF_VM >> ~/.bashrc
```

Para realizar esto es necesario que la máquina virtual tenga acceso a internet. En algunos casos se detectó que la máquina virtual no se conectaba automáticamente, para solucionar este problema, se ejecutan en terminal los siguientes comandos (sustituyendo ‘35’ por el número correspondiente):

Código 5.3: Solución conexión a internet

```
1 ifconfig -a  
2 sudo dhclient ens35
```

El siguiente diagrama ilustra las asignaciones correctas de variables de entorno (con direcciones IP falsas)

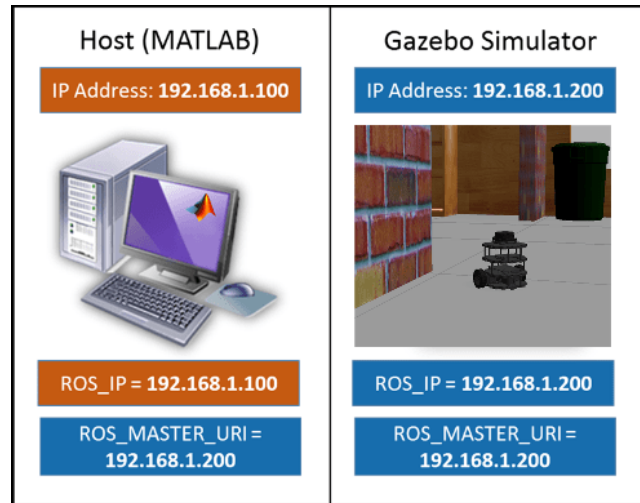


Figura 5.6: Diagrama de variables de entorno GazeboMatlab

Una vez configurada correctamente la red, se procede a abrir el ejemplo de ‘Gazebo House’.

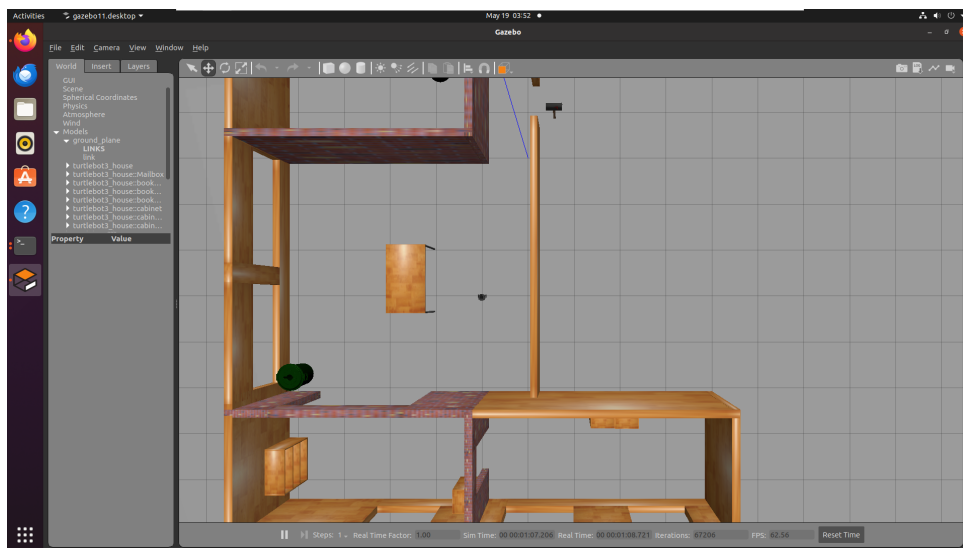


Figura 5.7: Gazebo House

5.4 Pruebas

Para comprobar el correcto funcionamiento, se deciden probar una serie de mensajes ROS básicos que permiten mover el TurtleBot, recibir su posición y orientación, y por último recibir los datos del *lidar*.

Para ello en la *Command Window* de MATLAB introducimos lo siguiente:

```
1 >> ipaddress = "http://192.168.197.128:11311";
2 >> rosinit(ipaddress)
```

Esto inicializará ROS, conectándose al TurtleBot que estamos simulando en Gazebo en la máquina virtual.

5.4.1 Velocidad

El mensaje tipo `‘/cmd_vel’` en ROS se utiliza para enviar comandos de velocidad al robot, especificando tanto la velocidad lineal como la angular que el robot debe seguir. Este mensaje, que suele ser publicado por nodos de control o de navegación, contiene información sobre la velocidad deseada en el espacio tridimensional, aunque comúnmente se utiliza en planos bidimensionales (velocidad lineal en los ejes x e y y velocidad angular alrededor del eje z).

```
1 >> velocity = 0.1;
2 >> robotCmd = rospublisher("/cmd_vel", "DataFormat", "struct") ;
3 >> velMsg = rosmessage(robotCmd);
4 >> velMsg.Linear.X = velocity;
5 >> send(robotCmd, velMsg)
6 >> pause(4);
7 >> velMsg.Linear.X = 0;
8 >> send(robotCmd, velMsg)
```

5.4.2 Posición

El mensaje tipo `‘/odom’` en ROS se utiliza para transmitir la información de odometría del robot, proporcionando datos esenciales sobre su posición y velocidad en el espacio. Este mensaje incluye la posición y orientación en coordenadas 3D del robot y su velocidad lineal y angular. Esta información permite implementar algoritmos de control y planificación del movimiento del robot.

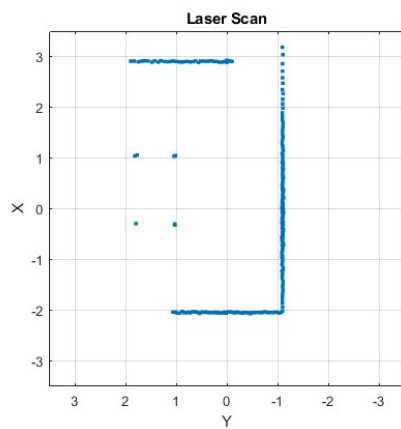
```
1 >> odomSub = rossubscriber("/odom", "DataFormat", "struct");
2 >> odomMsg = receive(odomSub, 3);
3 >> pose = odomMsg.Pose.Pose;
4 x = pose.Position.X;
5 y = pose.Position.Y;
6 z = pose.Position.Z;
7 >> [x y z]
8
9 ans =
10
11     -3.0002     1.0011    -0.0010
```

5.4.3 Lidar

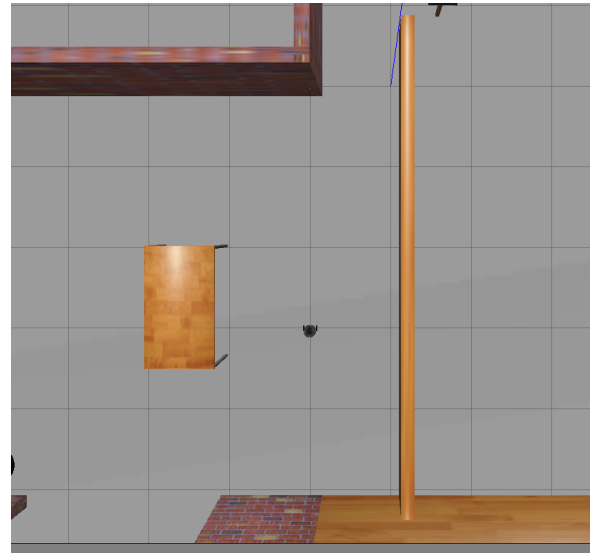
El mensaje tipo `‘/scan’` en ROS se utiliza para transmitir datos de sensores LiDAR. Este mensaje contiene información sobre la distancia medida a obstáculos en el entorno del robot, distribuida en un conjunto de ángulos. Los datos incluyen la distancia mínima y máxima detectada, el ángulo de inicio y final de escaneo, la resolución angular, y las intensidades de los retornos del láser si están disponibles. Estos datos son cruciales para tareas de navegación,

mapeo y evitar obstáculos, ya que permiten que el robot perciba y responda de manera efectiva a su entorno.

```
1 >> lidarSub = rossubscriber("/scan","DataFormat","struct");  
2 >> scanMsg = receive(lidarSub);  
3 >> rosPlot(scanMsg)
```



(a) Matlab



(b) Gazebo

Figura 5.8: Comparación datos lidar y simulador.



Capítulo 6

Control PC

6.1 Introducción

En este capítulo se describirán los subsistemas y modelos creados que permitirán el control del vehículo robótico desde un ordenador. Este ordenador se comunicará con la placa Arduino haciendo uso del puerto serie de esta. Para realizar esta comunicación se decide utilizar el nuevo *Add-On*, *Simulink-Real Time* introducido en la versión R2023b de MATLAB.

6.2 Pruebas iniciales comunicación serie.

Debido a la novedad de esta librería, se decide probar la comunicación con un programa simple que realice dos sumas y encienda dos LEDs.

El modelo de la figura 6.1 será el que se esté ejecutando en la placa Arduino. El bloque ‘*Protocol Encoder*’ recibirá cuatro señales, procedentes de dos constantes y su suma, y una última que corresponderá a la suma de los dos números recibidos. Para comprobar la recepción de estos se decide añadir dos bloques que detecten si la suma equivale a 5 ó 10 y encienda un LED conectado a dicho puerto. Una vez codificada la información esta se enviará por el puerto serie al ordenador sólo en el caso de que esta sea diferente. Se decide utilizar el ‘*Enabled Subsystem*’ para esto, con el objetivo de no saturar la comunicación.

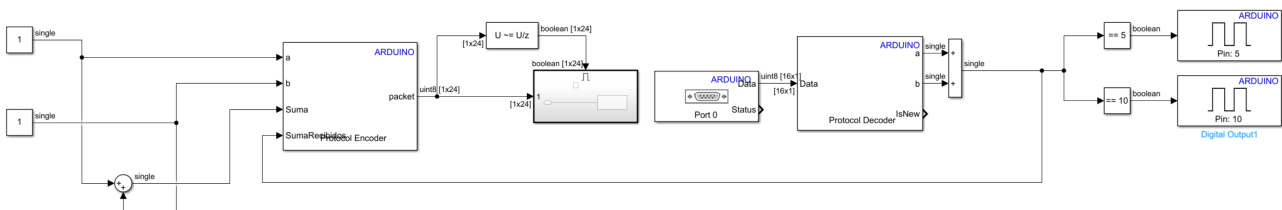


Figura 6.1: Modelo prueba serie Arduino.

[illegible]

Durante la realización de estas pruebas, se identificó la importancia de definir el tamaño del dato recibido por serial. Esta información es del tipo *uint8* y su tamaño lo podremos encontrar en la señal ya empaquetada que se pretende recibir. En este ejemplo el mensaje enviado al PC es de $[1 \times 24]$ y el enviado a la placa Arduino de $[1 \times 16]$.

Este subsistema recibirá como entradas los parámetros de velocidad lineal y angular así como una señal de *Mode* que determinará si queremos encender u apagar los motores. Esta información será codificada y enviada a la placa Arduino. Para no saturar el canal de comunicación sólo se transmitirá la información en caso de que esta haya cambiado.

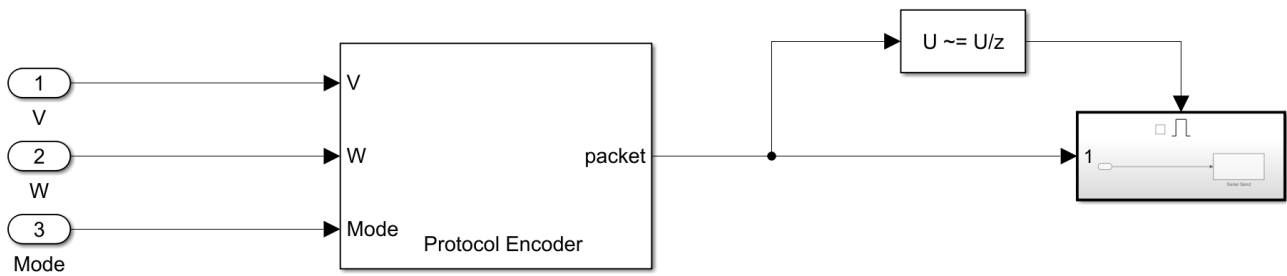
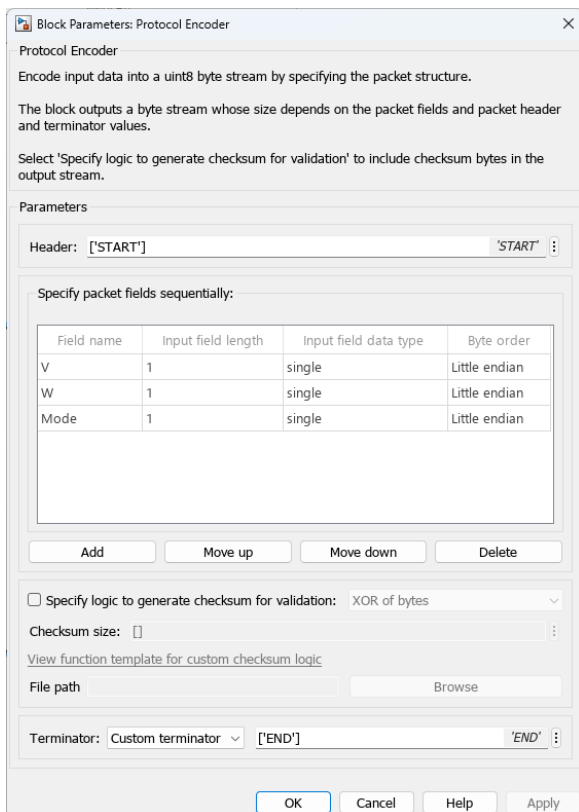
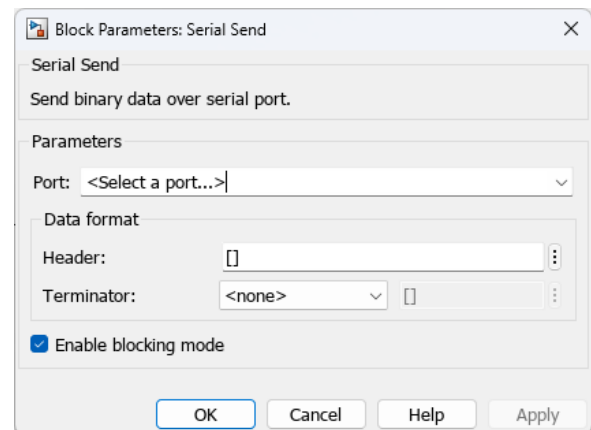


Figura 6.3: Subsistema Send to Arduino

Con el fin de unificar el método de codificación de las señales durante las comunicaciones en serie se decide que todos los datos sean del tipo *single* y que el orden de los bytes sea *Little endian*. Además todas las tramas de datos se iniciarán con ‘*START*’ y finalizarán con ‘*END*’. Estas configuraciones serán añadidas en el bloque ‘*Protocol Encoder*’ (Figura 6.4a), por último el se configurará el bloque ‘*Serial Send*’ para que transmita la información al puerto de comunicación en el que se encuentra conectada la placa Arduino, además no se utilizará ningún *Header* o *Terminator* puesto que estos ya habrán sido añadidos previamente.



(a) Protocol Encoder



(b) Serial Send

Figura 6.4: Parámetros de los bloques

6.3.2 *Receive from Arduino*

Este subsistema se encarga de recibir la información desde la placa, esta incluirá la velocidad lineal y angular y un booleano para motor que indicará la presencia o no de un error en estos. Dicha información se mostrará en los bloques ‘*Display*’ y a su vez será la que se muestre gráficamente en el modelo final (Figura 6.11).

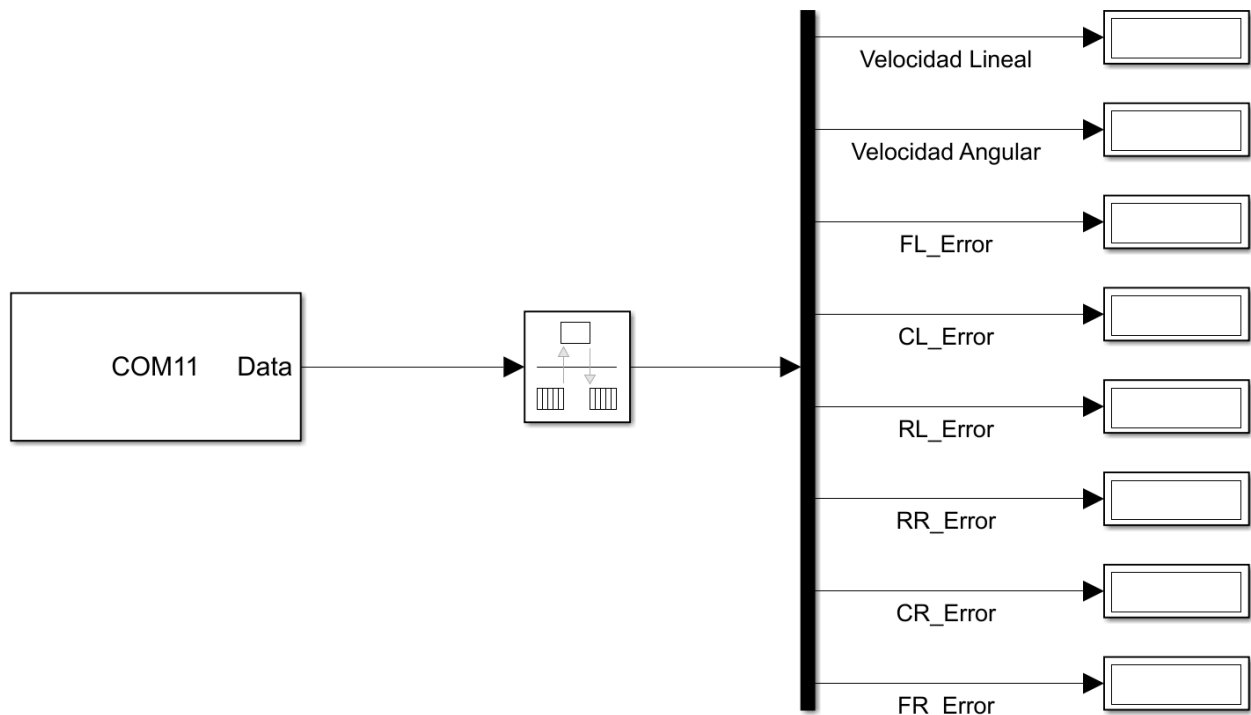


Figura 6.5: Subsistema Receive from Arduino

Debido a que el bloque ‘*Serial Receive*’ permite configurar el tipo y tamaño del dato recibido, además del *Header* y *Terminator*, no es necesario la utilización de un bloque dedicado íntegramente a la decodificación de la información.

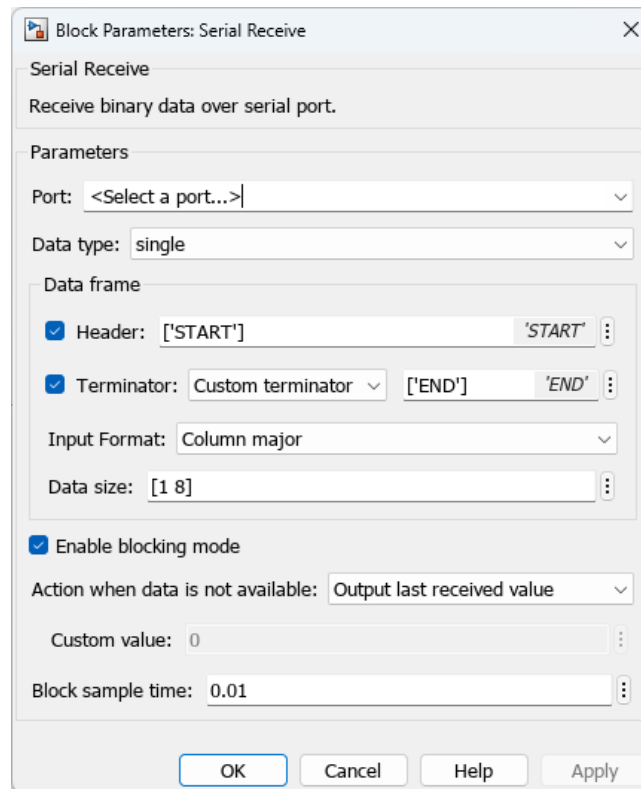


Figura 6.6: Parámetros del bloque Serial Receive

6.3.3 *XBox Controller*



Figura 6.7: Mando inalámbrico Xbox

Este subsistema permite controlar de manera inalámbrica el vehículo robótico haciendo uso de un mando de Microsoft (Figura 6.7). Su bloque principal, *XInput Controller*, se obtiene

a partir del *File Exchange* de MathWorks **SimulinkXboxController**. Este bloque informa tanto de las señales los botones y *joysticks*, como del estado de conexión y carga de la batería del mando.

Sus salidas son:

- *Axes*: Vector de seis señales procedentes de los ejes X e Y de cada *joystick* y de los gatillos (RT y LT). Estas señales son del tipo continuas, por lo que previo a su uso, se discretizan haciéndolas pasar por el bloque ‘Zero-Hold-Order’.
- *Buttons*: Vector de doce señales de activación (0 ó 1) procedentes de los botones del mando.
- *Info*: Vector de tres señales que proporciona información sobre el estado de la conexión bluetooth con el mando y del nivel de batería de este.

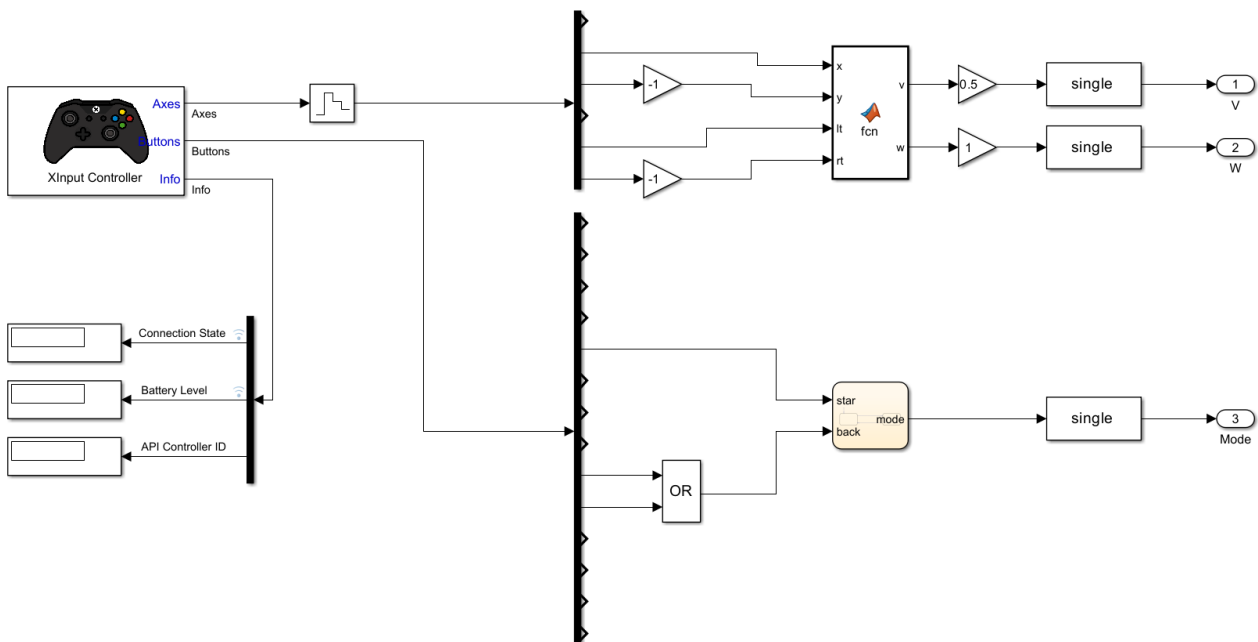


Figura 6.8: Subsistema XBox Controller

Para controlar el encendido y apagado de los motores, se hace uso del bloque ‘*Chart*’. Este nos permitirá definir los dos estados y las condiciones que se deben de cumplir para que el estado inicial (0) cambie.

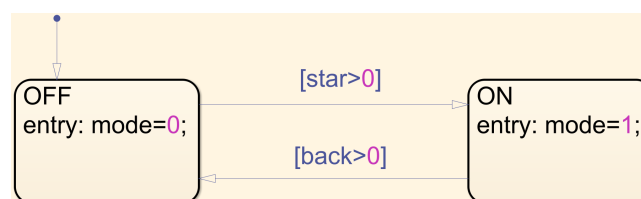


Figura 6.9: Contenido bloque ‘*chart*’

Inicialmente el motor se encontrará apagado. Este se encenderá al pulsar el botón ‘*START*’, y se apagará cuando se pulse uno de los botones ‘*RB*’ ó ‘*LB*’.

Por último, para realizar el cálculo de las velocidades linear y angular, se hace uso de la función de MATLAB ‘XBox - V y W’ cuyo código se incluye en el anexo C. Dicha función hará uso de las señales de los ejes X e Y provenientes de ambos *joysticks*.

6.3.4 ROS Publisher

Este subsistema se encarga de publicar en la red ROS correspondiente información a tiempo real sobre el estado del vehículo robótico. Este recibirá la velocidad linear y angular, y haciendo uso de esta, mediante el subsistema de odometría explicado más adelante (Figura 7.11), se publicará tanto las velocidades como la posición y orientación de este.

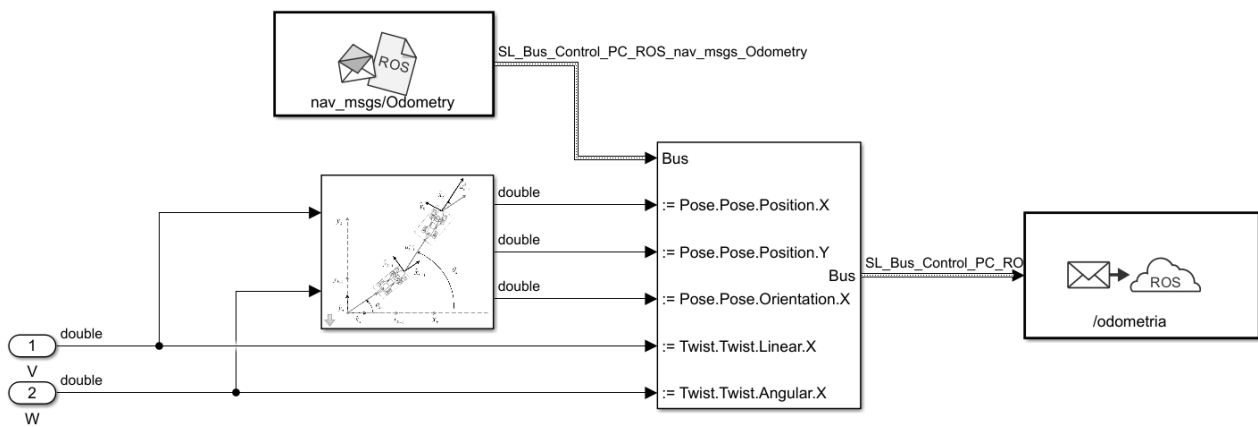


Figura 6.10: Subsistema ROS Publisher

6.4 Modelo control PC

Los subsistemas descritos previamente se integran en un modelo que permitirá el control manual del vehículo haciendo uso del mando. A este modelo se le añaden diversos indicadores que permitan conocer el estado del vehículo de manera sencilla.

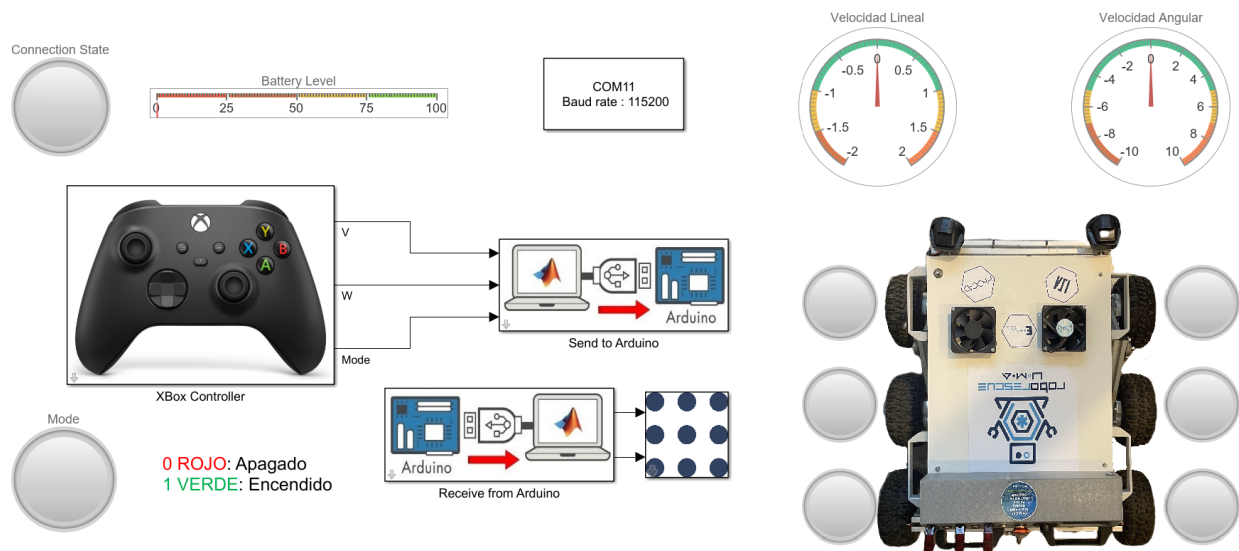


Figura 6.11: Modelo Control PC

Los indicadores añadidos son:

- 6 Lámparas para el vehículo: Estas se encenderán en verde cuando esté funcionando correctamente y cambiarán a rojo cuando se haya detectado algún error en el motor correspondiente.
- 1 Indicador Lineal: Conectado a la información del estado de carga de la batería del mando inalámbrico.
- 1 Lámpara: Encargada de indicar el estado de la conexión del mando (ROJO: Desconectado, VERDE: Conectado).
- 1 Lámpara: Permitirá visualizar si los motores están encendidos (VERDE) o apagados (ROJO).
- 2 Indicadores: Conectados a las señales de velocidad lineal y angular.

Capítulo 7

Control Vehículo

7.1 Introducción

En este capítulo se desarrollarán todos los subsistemas y modelos dedicados al control del vehículo robótico a través de la placa Arduino.

7.2 Subsistemas Simulink

7.2.1 *Receive from PC*

Este primer subsistema es el encargado de recibir la consigna de velocidad linear y angular, además del modo de funcionamiento desde el ordenador. El bloque ‘*Serial Receive*’ se configurará de manera que los datos recibidos sean del tipo *uint8* y con un tamaño de 20, esto viene determinado del ‘*Serial Send*’ del ordenador. A diferencia del modelo del PC en el que su *Serial Receive* permitía la determinación de las características de la trama de datos además del *Header* y *Terminator*, en este caso estos últimos no se pueden configurar, haciendo necesario el uso del bloque ‘*Protocol Decoder*’. Este bloque debe tener la misma configuración que el ‘*Protocol Encoder*’ del modelo del ordenador (Figura 6.4a).

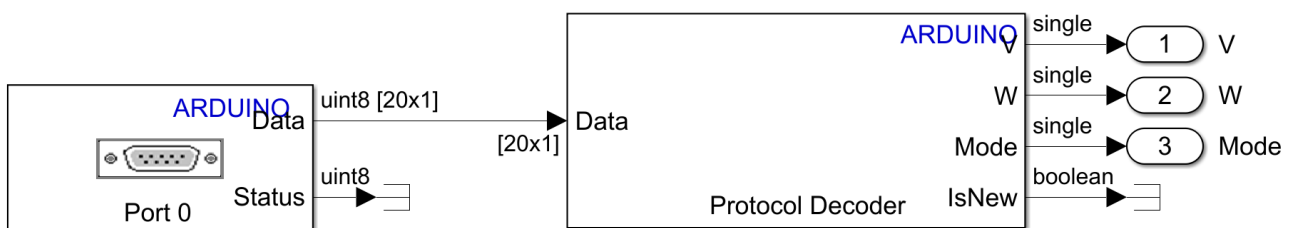
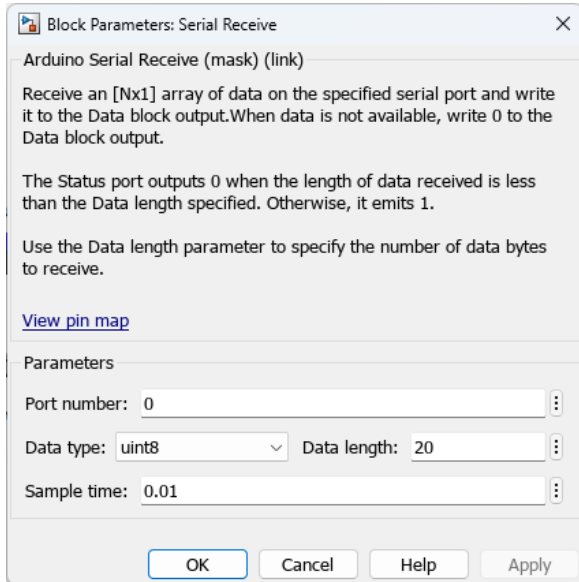
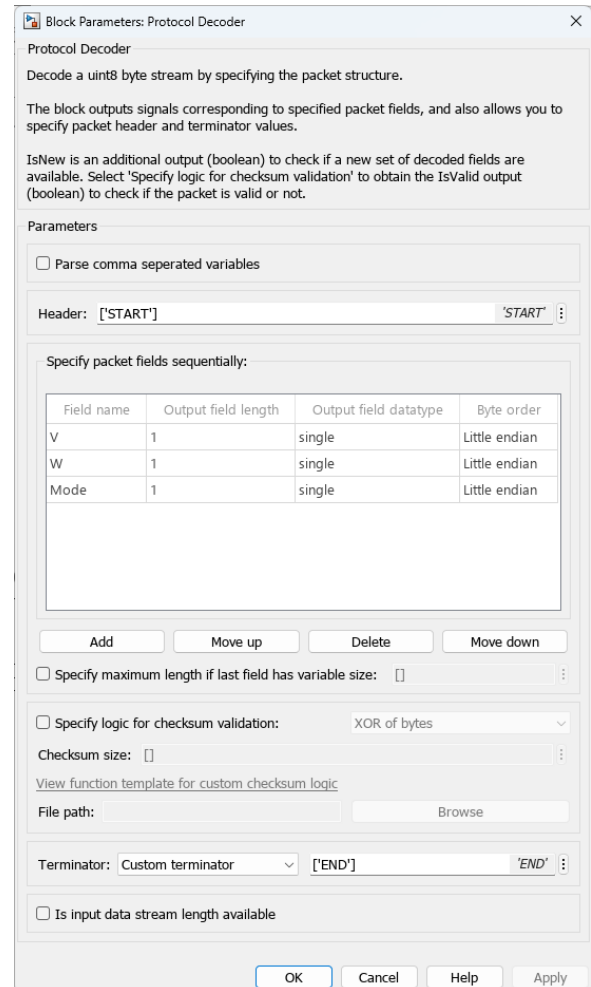


Figura 7.1: Subsistema *Receive from PC*



(a) Serial Receive



(b) Protocol Decoder

Figura 7.2: Parámetros de los bloques.

7.2.2 Modelo cinemático inverso

Este subsistema utiliza las distintas operaciones matemáticas necesarias para transformar las velocidades lineales, v , y angulares, ω del vehículo en las velocidades individuales de cada una de las seis ruedas. Para ello es necesario establecer una serie de parámetros que definan la geometría del chasis:

- R: Radio de ruedas.
- Lx: Distancia longitudinal desde el centro del chasis hasta las ruedas delanteras y traseras.
- Ly: Distancia lateral desde el centro del chasis hasta las ruedas laterales
- b: Ancho del chasis

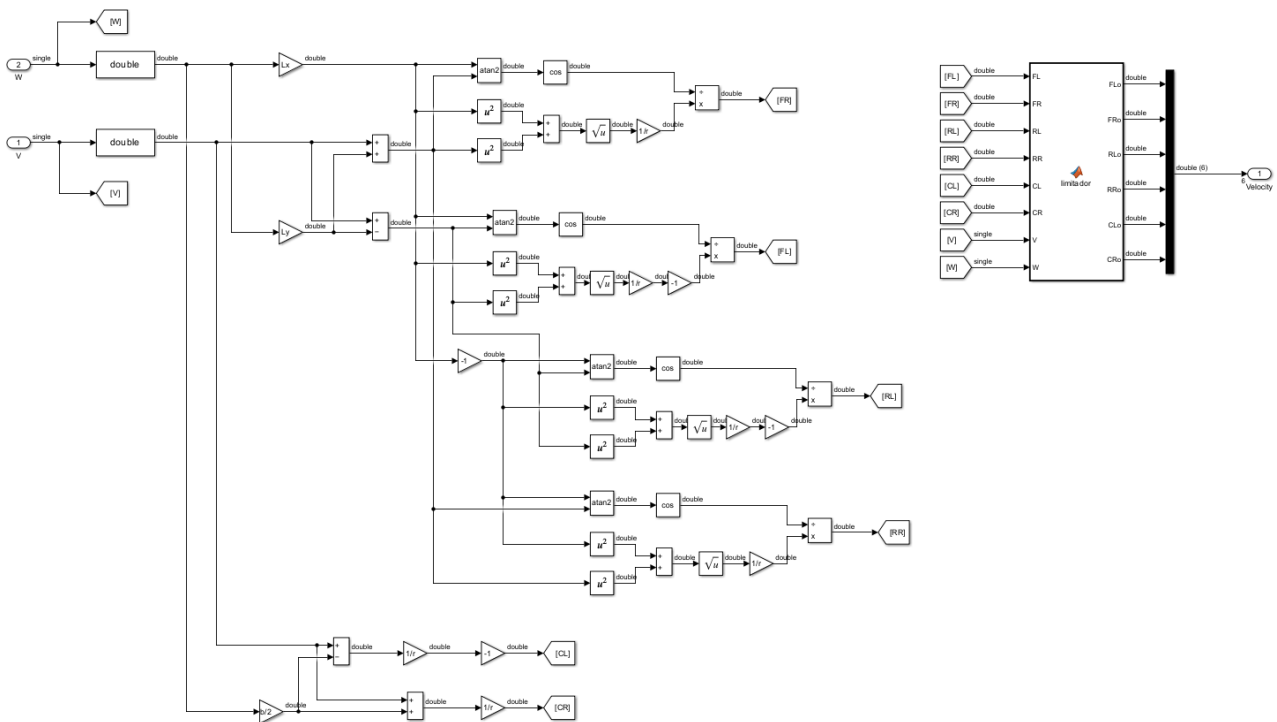


Figura 7.3: Subsistema Modelo Cinemático Inverso

En este mismo subsistema se añade una función de MATLAB, cuyo código ‘Limitador’ se encuentra en el anexo C. Esta función ajusta las velocidades de las ruedas del vehículo robótico en función de las entradas de velocidad lineal y angular. Si ambas velocidades son diferentes de cero, se corrigen las velocidades de las ruedas centrales y estas velocidades ajustadas se propagan a las ruedas delanteras y traseras. Si una de las velocidades es cero, se mantienen las velocidades originales.

7.2.3 Control Ruedas

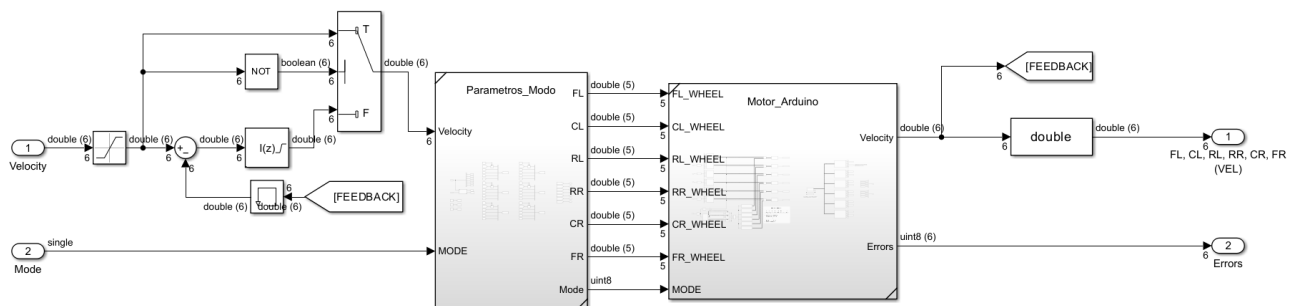


Figura 7.4: Subsistema Control Ruedas

Parámetros Modo

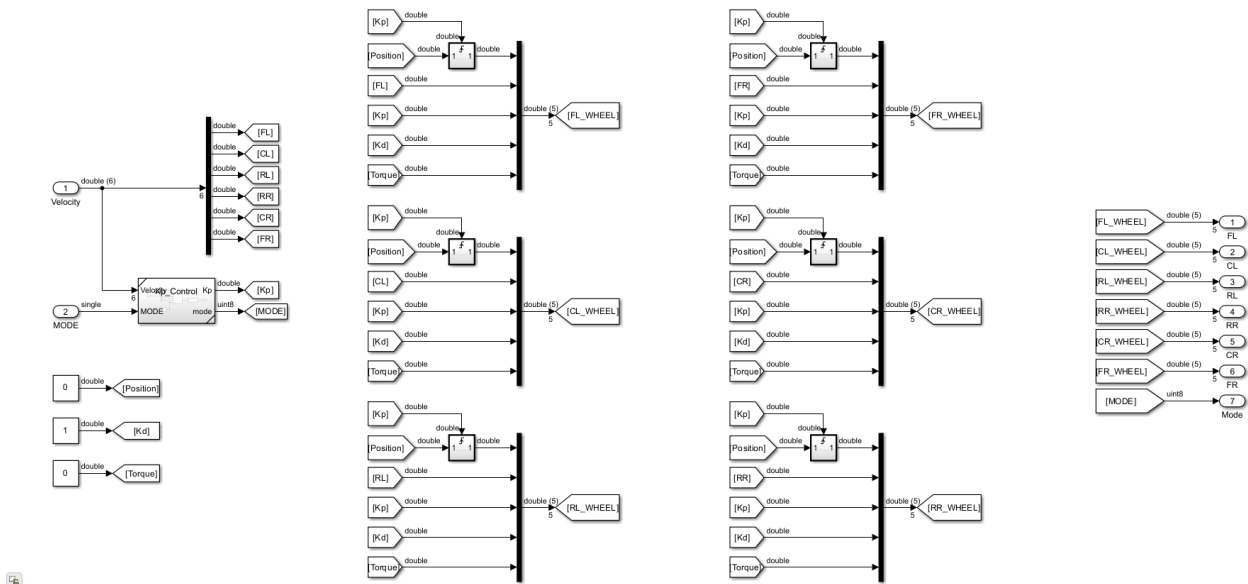


Figura 7.5: Subsistema Parámetros Modo

La función principal de este subsistema es crear los vectores que se utilizarán posteriormente para cada uno de los motores. Como entrada recibe el vector de seis velocidades correspondientes a cada una de las ruedas, y el modo de funcionamiento. Con esto se calcula K_p y el modo que finalmente se utilizará, haciendo uso del subsistema de control de K_p (Figura 7.6). Este subsistema tendrá seis salidas que corresponderán a cada uno de los motores y contendrán el vector de parámetros de control de estos.

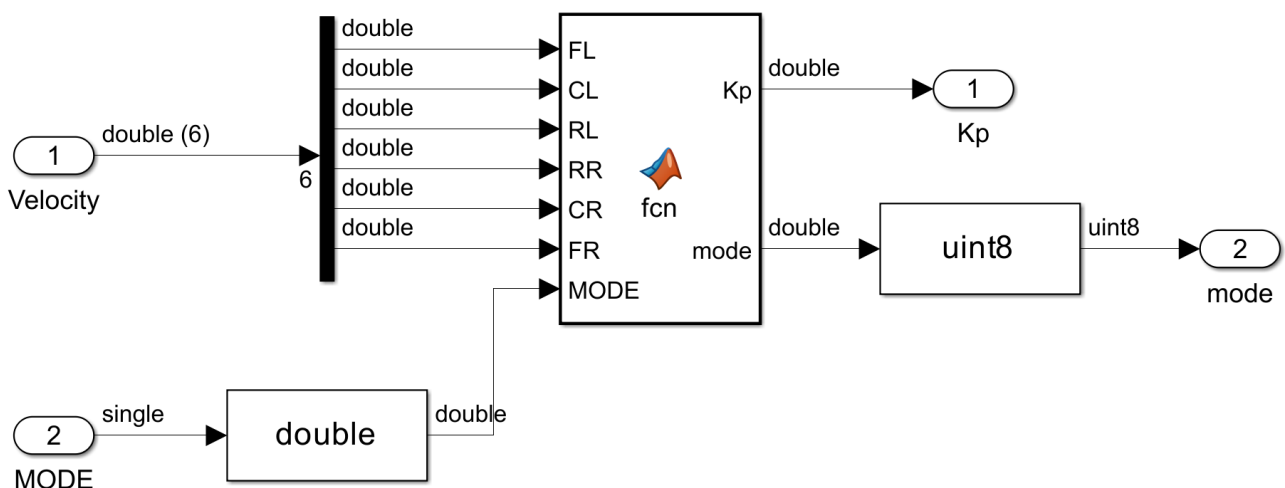


Figura 7.6: Subsistema Control K_p

El código del bloque ‘*MATLAB Function*’ se puede encontrar en el Apéndice C, este

determinará los valores de K_p y $mode$ en función de las velocidades y modo de funcionamiento de entrada. En caso de que todas las velocidades sean cero y el modo sea '1', K_p tendrá un valor de 25 y $mode$ cambiará a '2' (control de posición). En caso contrario K_p se establece a 0 y $mode$ se mantiene igual a la entrada.

Motor Arduino

Este subsistema es el encargado de comunicarse con las ruedas. Mandando las señales de control y recibiendo el estado de estos. El funcionamiento de dicho subsistema se desarrolló más en profundidad en apartados previos de la memoria.

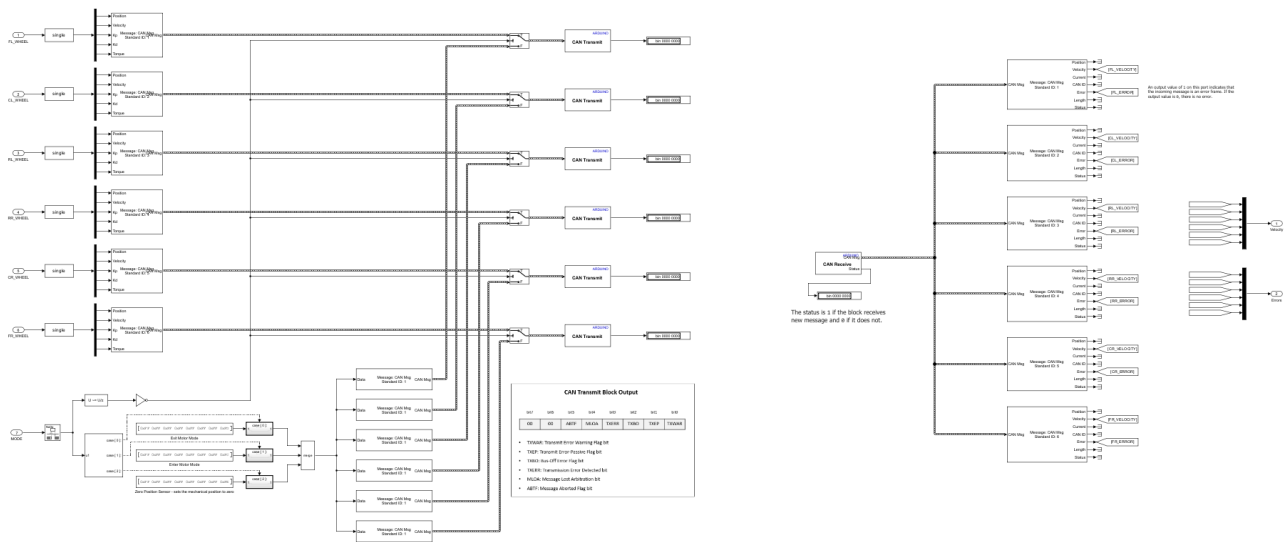


Figura 7.7: Subsistema Ruedas

En resumen, este subsistema tendrá siete entradas, seis de ellas correspondientes a los motores y una última de modo de funcionamiento. Las entradas perteneciente a los motores, son vectores de cinco parámetros: Posición, Velocidad, K_p , K_d y Torque. Estos serán empaquetados en un mensaje CAN y transmitidos al motor en todo momento, a excepción de los cambios de modo de funcionamiento. Por otro lado, este subsistema también recibe la información de los motores, desempaquetando esta para cada uno de los motores y sacando un vector de velocidades y otro de detección de errores.

7.2.4 Modelo cinemático directo

El modelo cinemático directo describe la relación entre las velocidades individuales de las ruedas y el movimiento global para el vehículo en términos de su velocidad lineal y angular. Se toman las ruedas centrales como motrices, sólo se utilizan las velocidades de estas para el cálculo.

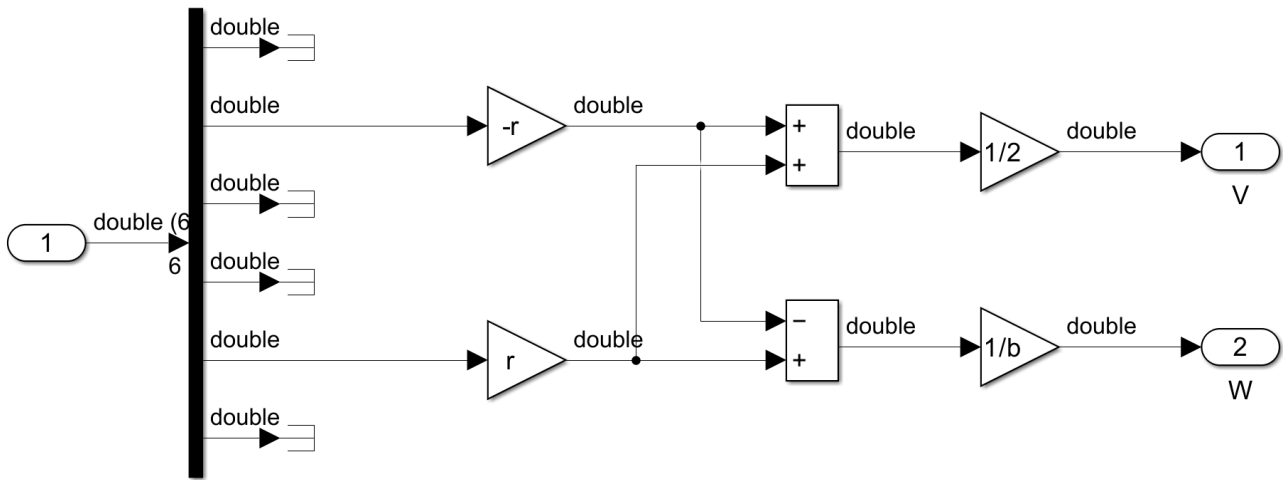


Figura 7.8: Subsistema Modelo Cinemático Directo

Este modelo implementa las siguientes ecuaciones:

$$v = \frac{R \times (v_{CR} - v_{CL})}{2} \quad (7.1)$$

$$v = \frac{R \times (v_{CR} + v_{CL})}{b} \quad (7.2)$$

7.2.5 Send to PC

La finalidad de este subsistema será la de transmitir la velocidad lineal y angular del vehículo al PC. La configuración del bloque ‘*Protocol Encoder*’ es hermana de la de, en este caso, el ‘*Serial Receive*’ del modelo del ordenador (Figura 6.6). Con la finalidad de no saturar el puerto de comunicación de la placa Arduino, el bloque ‘*Serial Transmit*’ se incluye dentro de un *Enabled Subsystem* que se activará únicamente cuando la información que se quiera transmitir haya cambiado.

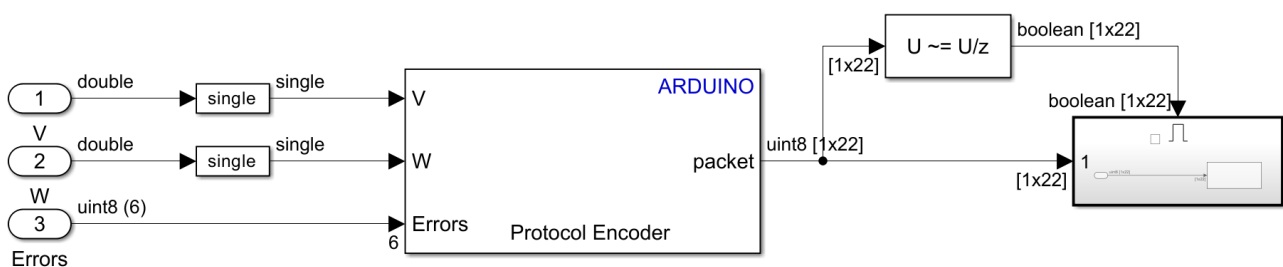
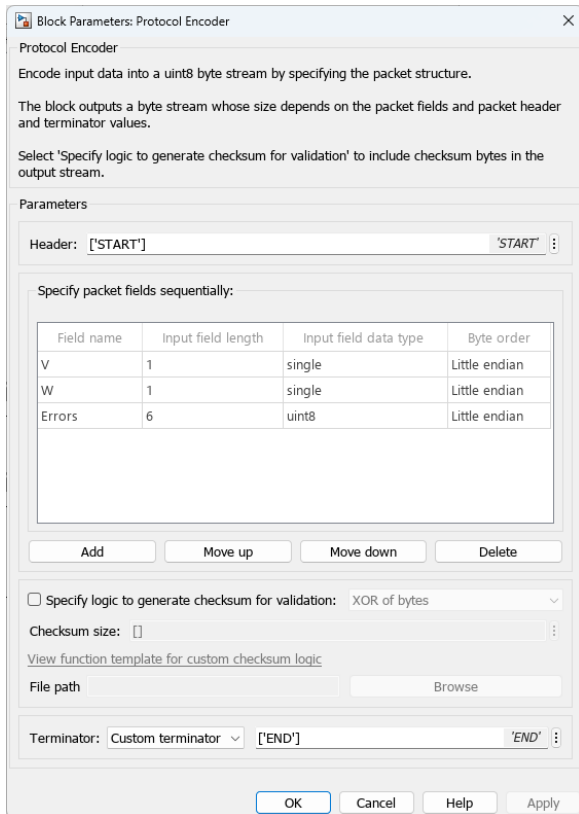


Figura 7.9: Subsistema Send to PC



Block Parameters: Protocol Encoder

Protocol Encoder

Encode input data into a uint8 byte stream by specifying the packet structure.

The block outputs a byte stream whose size depends on the packet fields and packet header and terminator values.

Select 'Specify logic to generate checksum for validation' to include checksum bytes in the output stream.

Parameters

Header: ["START"] 'START'

Specify packet fields sequentially:

Field name	Input field length	Input field data type	Byte order
V	1	single	Little endian
W	1	single	Little endian
Errors	6	uint8	Little endian

Add Move up Move down Delete

☐ Specify logic to generate checksum for validation: XOR of bytes

Checksum size: []

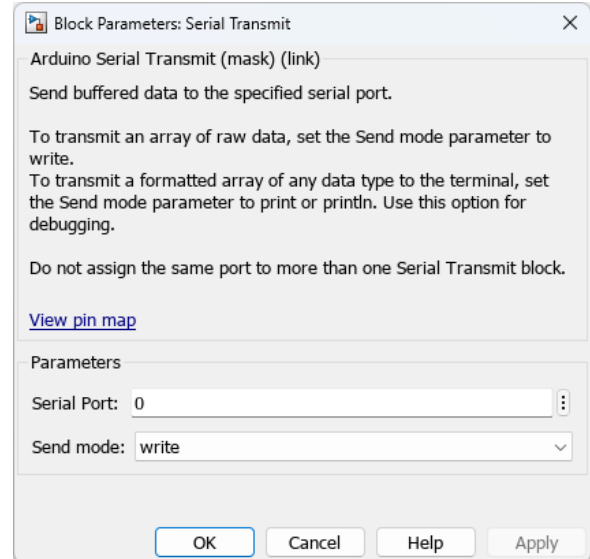
[View function template for custom checksum logic](#)

File path Browse

Terminator: Custom terminator ["END"] 'END'

OK Cancel Help Apply

(a) Protocol Encoder



Block Parameters: Serial Transmit

Arduino Serial Transmit (mask) (link)

Send buffered data to the specified serial port.

To transmit an array of raw data, set the Send mode parameter to write.

To transmit a formatted array of any data type to the terminal, set the Send mode parameter to print or println. Use this option for debugging.

Do not assign the same port to more than one Serial Transmit block.

[View pin map](#)

Parameters

Serial Port: 0

Send mode: write

OK Cancel Help Apply

(b) Serial Transmit

Figura 7.10: Parámetros de los bloques.

7.2.6 Odometría

Este subsistema calcula la posición y orientación del vehículo en función de sus velocidades lineal y angular y visualiza la trayectoria del vehículo en un plano XY .

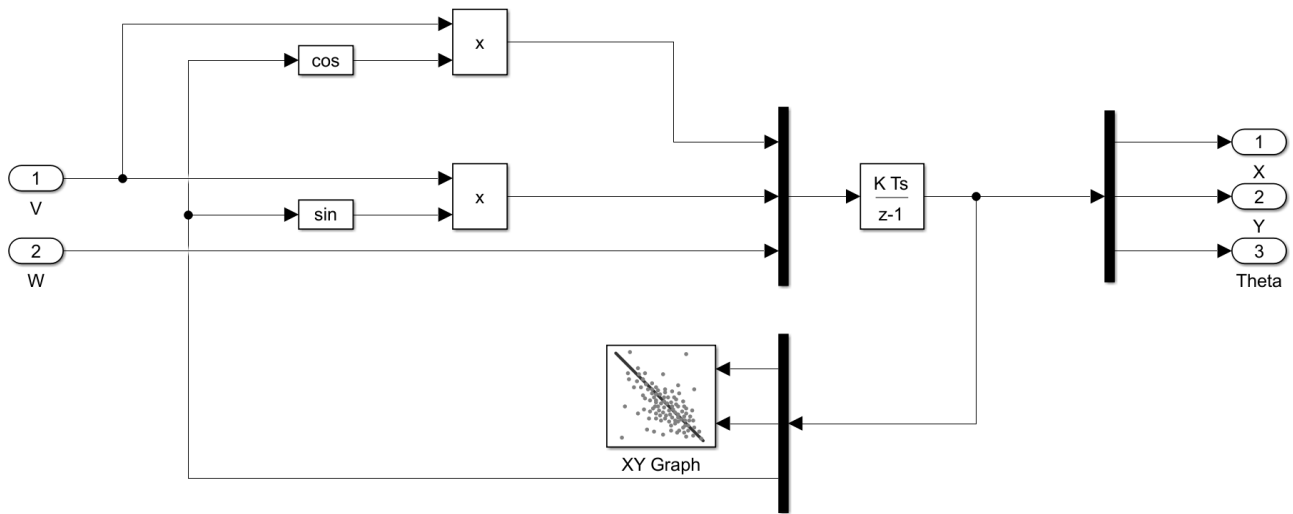


Figura 7.11: Subsistema Odometría

Las ecuaciones que definen este sistema son:

$$X = \int v_x dt = \int v \times \cos(\theta) dt \quad (7.3)$$

$$Y = \int v_y dt = \int v \times \sin(\theta) dt \quad (7.4)$$

$$\theta = \int \omega dt \quad (7.5)$$

donde:

- X : Posición del vehículo en el eje X.
- Y : Posición del vehículo en el eje Y.
- v_x : Velocidad lineal en el eje X.
- v_y : Velocidad lineal en el eje Y.
- θ : Ángulo de orientación del vehículo.

7.3 Modelo control vehículo

Los subsistemas descritos previamente serán los que integren este modelo. Su función será la de recibir la consigna de velocidad lineal y angular desde el ordenador, transformar esta en velocidades específicas para cada motor, dependiendo de estas se establecen los demás

parámetros de control de las ruedas y se transmiten. Partiendo de los motores se reciben las velocidades de cada uno que haciendo uso del modelo cinemático directo transformaremos en velocidad lineal y angular del vehículo, transmitiendo finalmente esta información de nuevo al ordenador.

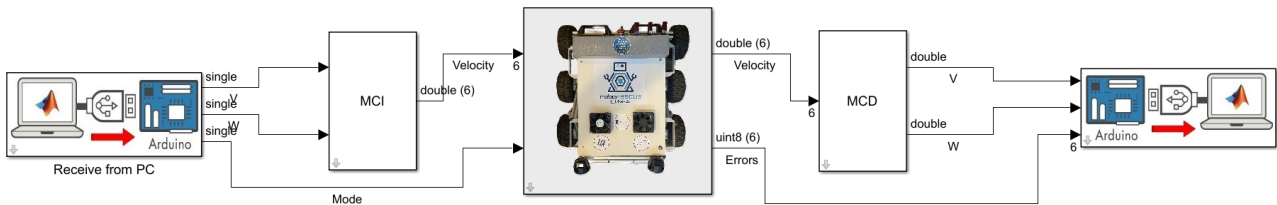


Figura 7.12: Modelo Control Vehículo



Capítulo 8

Conclusiones y mejoras futuras

8.1 Introducción

En este ultimo capítulo de la memoria se incluyen las conclusiones obtenidas durante el desarrollo del Trabajo de Fin de Grado, así como las posibles mejoras futuras para el proyecto.

8.2 Conclusiones

El objetivo principal de este Trabajo de Fin de Grado era el de implementar, haciendo uso de SIMULINK y una placa Arduino, un controlador para los servomotores del vehículo robótico Donatello.

Se ha desarrollado un sistema capaz de controlar los motores BLDC, haciendo uso de una Arduino Mega con su correspondiente CAN Shield, pese a la escasa información sobre estos servomotores proporcionada por el fabricante, se ha conseguido satisfactoriamente.

La utilización de la placa Arduino MKR WiFi 1010 ha presentado innumerables problemas en la generación de código por parte de MATLAB tanto en las versiones 2023b y 2024a, que la han descartado para su utilización en el robot.

La modificación de la comunicación en serie PC-Arduino haciendo uso de nuevas librerías de MathWorks publicadas para la versión R2023b, ha permitido prescindir de código C++ o S-Functions junto con sus dependencias, mejorando la comprensión y portabilidad del código fuente, facilitando las modificaciones futuras.

Se ha creado una versión de interfaz para ROS, lo cuál permite la utilización en el robot de los procedimientos ya existentes para el Robot Operating System.

Personalmente, me ha permitido adquirir conocimientos en el campo de la robótica móvil, además de avanzar en el uso de herramientas como MATLAB o Simulink. Como añadido, esta memoria ha sido realizada en Overleaf, un editor LaTeX, lo cuál me ha obligado a aprender este

lenguaje cada vez más utilizado para la redacción de textos y artículos científicos. Finalmente, este trabajo ha logrado apasionarme para seguir profundizando en el campo de la electrónica y automática.

8.3 Posibles mejoras

El presente TFG forma parte de un proyecto mayor en continuo desarrollo, siendo su objetivo final montar un vehículo robótico de rescate dotado de manipulador funcional. A continuación se detallan posibles incorporaciones en versiones futuras:

- Adición de unidades de medición inercial que permita calcular la posición, orientación y velocidad del vehículo sin necesidad de referencias externas, mejorando el control cinemático del vehículo.
- Insistir con futuras versiones de MATLAB en la sustitución de la Arduino Mega por otra placa que incorpore módulo de comunicación inalámbrica, eliminando comunicación serial cableada, haciendo más robusto el sistema.
- Integrar sensores de corriente para optimizar el consumo de energía y mejorar la duración de la carga de la batería.
- Incorporación de diversos sensores que permitan conocer con mayor precisión el entorno que rodea al vehículo.
- Sistema de Geolocalización que permita conocer la ubicación precisa del robot y la distancia al objetivo.
- Implementación de un modelo automático que sabiendo la localización actual y la objetivo, trace la trayectoria hacia este.

Apéndice A

ComunicacionMotores.ino

Código A.1: ComunicacionMotores.ino

```
1 #include <mcp_can.h>
2 #include <SPI.h>
3
4 #ifdef ARDUINO_SAMD_VARIANT_COMPLIANCE
5   #define SERIAL SerialUSB
6 #else
7   #define SERIAL Serial
8 #endif
9
10 const int SPI_CS_PIN = 3; // MKR ?
11
12 MCP_CAN CAN(SPI_CS_PIN);
13
14 unsigned char Enter_Motor_Mode[8] = {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFC};
15 unsigned char Exit_Motor_Mode[8] = {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFD};
16
17 unsigned char buf;
18 unsigned char len = 0;
19 long unsigned int canID = 0;
20 unsigned char ID_type = 0;
21
22 void setup() {
23   // put your setup code here, to run once:
24   SERIAL.begin(115200);
25 }
26
27
28 void loop() {
29   // put your main code here, to run repeatedly:
30
31   while(CAN_OK != CAN.begin(0, CAN_1000KBPS, MCP_8MHZ)) {
32     SERIAL.println("CAN BUS Shield init fail!");
33     SERIAL.println("Init CAN BUS Shield again");
34     delay(10);
35   }
36
37   SERIAL.println("CAN BUS Shield init ok!");
38
39   CAN.sendMsgBuf(0x05, 0, 8, Enter_Motor_Mode);
40
41   delay(1000);
42
43   if(CAN_MSGAVAIL == CAN.checkReceive()){
44     CAN.readMsgBuf(&canID, &ID_type, &len, &buf);
45     SERIAL.println(len);
46
47     SERIAL.println("-----");
48     SERIAL.println("Get data from ID: 0x");
49     SERIAL.println(canID, HEX);
50
51     SERIAL.println(buf, HEX);
52     SERIAL.println("\t");
53
54     SERIAL.println();
55   }
56
57   else
58     SERIAL.println("No llega nada");
59
60   delay(1000);
61
62 }
```


Apéndice B

CAN Master Test- Ben Katz

Código B.1: main.cpp

```
1 #define CAN_ID 0x1
2
3 #include "mbed.h"
4 #include "math_ops.h"
5
6
7 Serial          pc(PA_2, PA_3);
8 CAN             can(PB_8, PB_9, 1000000);    // CAN Rx pin name, CAN Tx pin name
9 CANMessage      rxMsg;
10 CANMessage      txMsg1;
11 CANMessage      txMsg2;
12 int             ledState;
13 Timer           timer;
14 Ticker          sendCAN;
15 int             counter = 0;
16 volatile bool   msgAvailable = false;
17 Ticker loop;
18
19 float theta1, theta2, dtheta1, dtheta2;
20
21 /// Value Limits ///
22 #define P_MIN -12.5f
23 #define P_MAX 12.5f
24 #define V_MIN -45.0f
25 #define V_MAX 45.0f
26 #define KP_MIN 0.0f
27 #define KP_MAX 500.0f
28 #define KD_MIN 0.0f
29 #define KD_MAX 5.0f
30 #define T_MIN -18.0f
31 #define T_MAX 18.0f
32
33 /// CAN Command Packet Structure ///
34 /// 16 bit position command, between -4*pi and 4*pi
35 /// 12 bit velocity command, between -30 and + 30 rad/s
36 /// 12 bit kp, between 0 and 500 N-m/rad
37 /// 12 bit kd, between 0 and 100 N-m*s/rad
38 /// 12 bit feed forward torque, between -18 and 18 N-m
39 /// CAN Packet is 8 8-bit words
40 /// Formatted as follows. For each quantity, bit 0 is LSB
41 /// 0: [position[15-8]]
42 /// 1: [position[7-0]]
43 /// 2: [velocity[11-4]]
44 /// 3: [velocity[3-0], kp[11-8]]
45 /// 4: [kp[7-0]]
46 /// 5: [kd[11-4]]
47 /// 6: [kd[3-0], torque[11-8]]
48 /// 7: [torque[7-0]]
49
50 void pack_cmd(CANMessage * msg, float p_des, float v_des, float kp, float kd, float t_ff){
51     /// limit data to be within bounds ///
52     p_des = fminf(fmaxf(P_MIN, p_des), P_MAX);
53     v_des = fminf(fmaxf(V_MIN, v_des), V_MAX);
54     kp = fminf(fmaxf(KP_MIN, kp), KP_MAX);
55     kd = fminf(fmaxf(KD_MIN, kd), KD_MAX);
56     t_ff = fminf(fmaxf(T_MIN, t_ff), T_MAX);
57     /// convert floats to unsigned ints ///
58     int p_int = float_to_uint(p_des, P_MIN, P_MAX, 16);
59     int v_int = float_to_uint(v_des, V_MIN, V_MAX, 12);
60     int kp_int = float_to_uint(kp, KP_MIN, KP_MAX, 12);
61     int kd_int = float_to_uint(kd, KD_MIN, KD_MAX, 12);
62     int t_int = float_to_uint(t_ff, T_MIN, T_MAX, 12);
63     /// pack ints into the can buffer ///
64     msg->data[0] = p_int>>8;
65     msg->data[1] = p_int&0xFF;
66     msg->data[2] = v_int>>4;
67     msg->data[3] = ((v_int&0xF)<<4)|(kp_int>>8);
```

```
68     msg->data[4] = kp_int&0xFF;
69     msg->data[5] = kd_int>>4;
70     msg->data[6] = ((kd_int&0xF)<<4)|(t_int>>8);
71     msg->data[7] = t_int&0xFF;
72 }
73
74 /// CAN Reply Packet Structure ///
75 /// 16 bit position, between -4*pi and 4*pi
76 /// 12 bit velocity, between -30 and + 30 rad/s
77 /// 12 bit current, between -40 and 40;
78 /// CAN Packet is 5 8-bit words
79 /// Formatted as follows. For each quantity, bit 0 is LSB
80 /// 0: [position[15-8]]
81 /// 1: [position[7-0]]
82 /// 2: [velocity[11-4]]
83 /// 3: [velocity[3-0], current[11-8]]
84 /// 4: [current[7-0]]
85
86 void unpack_reply(CANMessage msg){
87     /// unpack ints from can buffer ///
88     int id = msg.data[0];
89     int p_int = (msg.data[1]<<8)|msg.data[2];
90     int v_int = (msg.data[3]<<4)|(msg.data[4]>>4);
91     int i_int = ((msg.data[4]&0xF)<<8)|msg.data[5];
92     /// convert ints to floats ///
93     float p = uint_to_float(p_int, P_MIN, P_MAX, 16);
94     float v = uint_to_float(v_int, V_MIN, V_MAX, 12);
95     float i = uint_to_float(i_int, -I_MAX, I_MAX, 12);
96
97     if(id == 2){
98         theta1 = p;
99         dtheta1 = v;
100     }
101     else if(id == 3){
102         theta2 = p;
103         dtheta2 = v;
104     }
105
106     //printf("%d  %.3f  %.3f  %.3f\n\r", id, p, v, i);
107 }
108
109 void onMsgReceived() {
110     can.read(rxMsg); // read message into Rx message storage
111     unpack_reply(rxMsg);
112 }
113
114
115 void sendCMD(){
116     /// bilateral teleoperation demo ///
117     pack_cmd(&txMsg1, 0, 0, 0, 0, -0.1f);
118     //pack_cmd(&txMsg2, theta1, dtheta1, 10, .1, 0);
119     //pack_cmd(&txMsg2, 0, 0, 0, 0, 1.0);
120     //pack_cmd(&txMsg1, 0, 0, 0, 0, 1.0);
121     //can.write(txMsg2);
122     wait(.0003); // Give motor 1 time to respond.
123     can.write(txMsg1);
124 }
125
126 void serial_isr(){
127     /// handle keyboard commands from the serial terminal ///
128     while(pc.readable()){
129         char c = pc.getc();
130         switch(c){
131             case(27):
132                 printf("\n\r exiting motor mode \n\r");
133                 txMsg1.data[0] = 0xFF;
134                 txMsg1.data[1] = 0xFF;
135                 txMsg1.data[2] = 0xFF;
136                 txMsg1.data[3] = 0xFF;
```

```
137         txMsg1.data[4] = 0xFF;
138         txMsg1.data[5] = 0xFF;
139         txMsg1.data[6] = 0xFF;
140         txMsg1.data[7] = 0xFD;
141
142         txMsg2.data[0] = 0xFF;
143         txMsg2.data[1] = 0xFF;
144         txMsg2.data[2] = 0xFF;
145         txMsg2.data[3] = 0xFF;
146         txMsg2.data[4] = 0xFF;
147         txMsg2.data[5] = 0xFF;
148         txMsg2.data[6] = 0xFF;
149         txMsg2.data[7] = 0xFD;
150         break;
151     case('m'):
152         printf("\n\r entering motor mode \n\r");
153         txMsg1.data[0] = 0xFF;
154         txMsg1.data[1] = 0xFF;
155         txMsg1.data[2] = 0xFF;
156         txMsg1.data[3] = 0xFF;
157         txMsg1.data[4] = 0xFF;
158         txMsg1.data[5] = 0xFF;
159         txMsg1.data[6] = 0xFF;
160         txMsg1.data[7] = 0xFC;
161
162         txMsg2.data[0] = 0xFF;
163         txMsg2.data[1] = 0xFF;
164         txMsg2.data[2] = 0xFF;
165         txMsg2.data[3] = 0xFF;
166         txMsg2.data[4] = 0xFF;
167         txMsg2.data[5] = 0xFF;
168         txMsg2.data[6] = 0xFF;
169         txMsg2.data[7] = 0xFC;
170         break;
171     case('z'):
172         printf("\n\r zeroing \n\r");
173         txMsg1.data[0] = 0xFF;
174         txMsg1.data[1] = 0xFF;
175         txMsg1.data[2] = 0xFF;
176         txMsg1.data[3] = 0xFF;
177         txMsg1.data[4] = 0xFF;
178         txMsg1.data[5] = 0xFF;
179         txMsg1.data[6] = 0xFF;
180         txMsg1.data[7] = 0xFE;
181
182         txMsg2.data[0] = 0xFF;
183         txMsg2.data[1] = 0xFF;
184         txMsg2.data[2] = 0xFF;
185         txMsg2.data[3] = 0xFF;
186         txMsg2.data[4] = 0xFF;
187         txMsg2.data[5] = 0xFF;
188         txMsg2.data[6] = 0xFF;
189         txMsg2.data[7] = 0xFE;
190         break;
191     }
192 }
193 can.write(txMsg1);
194 can.write(txMsg2);
195
196 }
197
198 int can_packet[8] = {1, 2, 3, 4, 5, 6, 7, 8};
199 int main() {
200     pc.baud(921600);
201     pc.attach(&serial_isr);
202     can.attach(&onMsgReceived); // attach 'CAN receive-complete' interrupt handler
203     can.filter(CAN_ID<<21, 0xFFE00004, CANStandard, 0); //set up can filter
204     printf("Master\n\r");
205     //printf("%d\n\r", RX_ID << 18);
```

```
206     int count = 0;
207     txMsg1.len = 8;           //transmit 8 bytes
208     txMsg2.len = 8;           //transmit 8 bytes
209     rxMsg.len = 6;            //receive 5 bytes
210     loop.attach(&sendCMD, .001);
211     txMsg1.id = 0x2;           //1st motor ID
212     txMsg2.id = 0x3;           //2nd motor ID
213     pack_cmd(&txMsg1, 0, 0, 0, 0, 0); //Start out by sending all 0's
214     pack_cmd(&txMsg2, 0, 0, 0, 0, 0);
215     timer.start();
216
217     while(1) {
218     }
219
220
221 }
```

Código B.2: math_ops.cpp

```
1 #include "math_ops.h"
2
3
4 float fmaxf(float x, float y){
5     /// Returns maximum of x, y ///
6     return (((x)>(y))?(x):(y));
7 }
8
9 float fminf(float x, float y){
10    /// Returns minimum of x, y ///
11    return (((x)<(y))?(x):(y));
12 }
13
14 float fmaxf3(float x, float y, float z){
15    /// Returns maximum of x, y, z ///
16    return (x > y ? (x > z ? x : z) : (y > z ? y : z));
17 }
18
19 float fminf3(float x, float y, float z){
20    /// Returns minimum of x, y, z ///
21    return (x < y ? (x < z ? x : z) : (y < z ? y : z));
22 }
23
24 void limit_norm(float *x, float *y, float limit){
25    /// Scales the lenght of vector (x, y) to be ≤ limit ///
26    float norm = sqrt(*x * *x + *y * *y);
27    if(norm > limit){
28        *x = *x * limit/norm;
29        *y = *y * limit/norm;
30    }
31 }
32
33
34 int float_to_uint(float x, float x_min, float x_max, int bits){
35    /// Converts a float to an unsigned int, given range and number of bits ///
36    float span = x_max - x_min;
37    float offset = x_min;
38    return (int) ((x - offset) * ((float)((1<<bits) - 1)) / span);
39 }
40
41
42 float uint_to_float(int x_int, float x_min, float x_max, int bits){
43    /// converts unsigned int to float, given range and number of bits ///
44    float span = x_max - x_min;
45    float offset = x_min;
46    return ((float)x_int) * span / ((float)((1<<bits) - 1)) + offset;
47 }
```




Apéndice C

MATLAB Functions

Código C.1: XBox - V y W

```
1 function [v,w]=fcn(x,y,lt,rt)
2
3     if (rt==0) && (lt==0) && ((x>0.3) || (x<-0.3))
4         v=x;
5         w=y;
6     elseif (x≠0) && ((rt≠0) || (lt≠0))
7         w=0;
8         v=0;
9     elseif (x==0) && (y==0) && (rt==0) && (lt≠0)
10        w=lt;
11        v=0;
12    elseif (x==0) && (y==0) && (lt==0) && (rt≠0)
13        w=rt;
14        v=0;
15    elseif (lt≠0) && (rt≠0)
16        w=0;
17        v=0;
18    elseif (x==0) || (y==0) || (lt==0)
19        w=lt;
20        v=0;
21    else
22        w=0;
23        v=0;
24    end;
```

Código C.2: MCI - Limitador

```
1 function [FLo,FRo,RLo,RRo,CLo,CRo] = limitador(FL,FR,RL,RR,CL,CR,V,W)
2
3 double CLi;
4 double CRi;
5
6 if ((V≠0) && (W≠0))
7
8     CLi=CL;
9     CRi=CR;
10
11     if ((V>0) && (CRi<0) || (V<0) && (CRi>0))
12         CRi=0;
13     elseif ((V>0) && (CLi>0) || (V<0) && (CLi<0))
14         CLi=0;
15     end
16
17     CLo=CLi;
18     CRo=CRi;
19     FLo=CLi;
20     RLo=CLi;
21     FRo=CRi;
22     RRo=CRi;
23
24 else
25
26     FLo=FL;
27     RLo=RL;
28     FRo=FR;
29     RRo=RR;
30     CLo=CL;
31     CRo=CR;
32
33
34 end
```

Código C.3: Kp_Control



```
1 function [Kp, mode] = fcn(FL,CL,RL,RR,CR,FR,MODE)
2
3     if (FL==0) && (CL==0) && (RL==0) && (RR==0) && (CR==0) && (FR==0) && (MODE==1)
4         Kp = 25;
5         mode = 2;
6     else
7         Kp = 0;
8         mode = MODE;
9     end
```

